



METODOLOGÍA PARA IMPLEMENTAR UN MODELO HILES DE UNA WSN

Autor

Oscar Javier Idrobo López

Tutor

Gerardo Muñoz

Universidad Distrital Francisco José De Caldas

Maestría en Ciencias de la Información y las Comunicaciones

Énfasis en Ingeniería de Software

Bogotá, Colombia

Octubre de 2024

CONTENIDO

TABLA DE FIGURAS	4
RELACIÓN DE TABLAS	1
1. RESUMEN.....	2
2. INTRODUCCION.....	2
2.1 FORMULACION DEL PROBLEMA.....	4
2.1.1 SISTEMATIZACION DEL PROBLEMA.....	4
2.2 OBJETIVOS	5
2.2.1 OBJETIVO GENERAL.....	5
2.2.2 OBJETIVOS EXPECIFICOS	5
2.3 ACTIVIDADES A DESARROLLAR	5
3. MARCO TEORICO.....	6
3.1 RED INALAMBRICA DE SENSORES O WSN	6
3.1.1 PROTOCOLOS DE COMUNICACION.....	7
3.1.1.1 PROTOCOLO LORA.....	8
3.1.1.2 PROTOCOLO ZIGBEE	9
3.1.1.3 PROTOCOLO WIFI	10
3.1.1.4 PROTOCOLO SIGFOX.....	10
3.2 FORMALISMO HILES.....	11
3.3 DISPOSITIVO SYSTEM ON CHIP.....	20
4. ACTIVIDADES DESARROLLADAS	21
4.1 PLANTEAMIENTO DE WSN Y MODELAMIENTO FORMAL	21
4.2 REPRESENTACION TEXTUAL PARA LOS ELEMENTOS HILES	27
4.3 REPRESENTACION VISUAL DE LOS ELEMENTOS HILES	28

4.4	EVALUACION DE LOS DISPOSITIVOS DE IMPLEMENTACION	30
4.4.1	ESP8266	30
4.4.2	ESP32	33
4.4.3	PRODUCTOS DIGI XBEE.....	36
4.4.4	RASPBERRY PI ZERO W.....	39
4.4.5	RASPBERRY PI PICO	40
4.5	HERRAMIENTA PARA LA CONSTRUCCION DE ARCHIVOS DE SIMULACION E IMPLEMENTABLES.	41
4.5.1	ESTRUCTURA “ALLELEMENTS” OBJETO JSON.....	42
4.5.2	ESTRUCTURA “RELATIONNODE” OBJETO JSON	44
4.5.3	ESTRUCTURA “NETWORK” OBJETO JSON	45
4.6	SIMULACION LOGICA DEL SISTEMA.....	51
4.6.1	SIMULACION LOGICA DEL NODO DE HUMEDAD.....	51
4.6.2	SIMULACION LOGICA DEL NODO DE RIEGO	52
4.7	VALIDACION DEL SISTEMA EN DISPOSITIVOS FISICOS	54
5.	CONCLUSIONES.....	57
6.	PRINCIPALES APORTES	58
7.	RECOMENDACIONES Y TRABAJOS FUTUROS.....	59
8.	REFERENCIAS.....	60

TABLA DE FIGURAS

Figura 1.	Capas protocolo LoRa	8
Figura 2.	Capas protocolo Zigbee	10
Figura 3.	Bloque estructural – HiLeS	12
Figura 4.	Bloque estructural sensor de humedad	12
Figura 5.	Bloque estructural que representa un nodo en una WSN	13
Figura 6.	Bloque funcional – HiLeS.....	13
Figura 7.	Bloque funcional para control de humedad mayor a 76%	14
Figura 8.	Canal continuo	14
Figura 9.	Canal evento discreto	14
Figura 10.	Canal Red de Petri.....	14
Figura 11.	Servicios HiLeS.....	15
Figura 12.	Puertos - HiLeS.....	15
Figura 13.	Nivel jerárquico uno	16
Figura 14.	Múltiples capas jerárquicas.....	17
Figura 15.	Sample and Holding A	17
Figura 16.	Sample and Holding B	17
Figura 17.	Uso de operador Sample en un WSN.....	18
Figura 18.	Representación de una transición en una red de Petri.....	19
Figura 19.	Representación del elemento lugar en una red de Petri.....	19
Figura 20.	Representación de un token en una red de Petri	19
Figura 21.	Red de Petri para encendido y apagado para un nodo de riego.	20
Figura 22.	Arquitectura SoC.....	21
Figura 23.	Red Inalámbrica de sensores para planta	22
Figura 24.	Diagrama de requerimientos ciclo de vida planta	22
Figura 25.	Diagrama definición de bloques.....	23
Figura 26.	Diagrama paramétrico sensor de humedad.....	23
Figura 27.	Diagrama máquina de estados Transmisor	24
Figura 28.	Diagrama máquina de estados Receptor	25
Figura 29.	Implementación HiLes Red Inalámbrica ejemplo.....	26

Figura 30.	Meta Modelo HiLeS	27
Figura 31.	Resultado descripción textual de sistema HiLeS	30
Figura 32.	Diagrama de bloques ESP8266EX	31
Figura 33.	Especificaciones Técnicas ESP8266EX Parte 1	32
Figura 34.	Especificaciones Técnicas ESP8266EX Parte 2	33
Figura 35.	Diagrama de bloque funcional ESP32	34
Figura 36.	Esquema Digi XBee	37
Figura 37.	Especificaciones técnicas XBee PRO S2C	38
Figura 38.	Review Raspberry Pi Zero	40
Figura 39.	Diagrama de componentes para la creación de código	42
Figura 40.	Estructura objeto JSON definición del sistema	42
Figura 41.	Objeto JSON bloque estructural HiLes nodo de humedad	43
Figura 42.	Definición de servicios en Objetos HiLes	43
Figura 43.	Objeto JSON nodo de humedad campo "relationNode"	44
Figura 44.	Objeto JSON nodo de humedad campo "relationNode"	45
Figura 45.	Estructura objeto JSON campo "network"	45
Figura 46.	Estructura de paquetes para la construcción de archivos .py	46
Figura 47.	Paquete "architecture"	46
Figura 48.	Paquete "configuration"	46
Figura 49.	Paquete "domain"	47
Figura 50.	Paquete "generator"	47
Figura 51.	Paquete "generator_files"	48
Figura 52.	Paquete "sdk"	48
Figura 53.	Paquete "utils"	49
Figura 54.	Ejemplo de código generado para el nodo de humedad	49
Figura 55.	Importación sdk "simulate"" en nodo etapa de simulación	50
Figura 56.	Importación sdk "real"" en nodo, para cargue en dispositivo físico	50
Figura 57.	Ejemplo de código generado nodo de riego	50
Figura 58.	Simulación nodo de humedad	51
Figura 59.	Representación textual generador de señal en nodo de humedad	52
Figura 60.	Simulación nodo de riego	53
Figura 61.	Representación textual señal "status" en el nodo de visualización	54

RELACIÓN DE TABLAS

Tabla 1.	Representación Textual y visual de los elementos HiLeS.....	29
Tabla 2.	Especificación Técnicas ESP32	36
Tabla 3.	Validación de requerimiento estado uno	55
Tabla 4.	Validación de requerimiento estado dos	55
Tabla 5.	Validación de requerimiento estado tres	55
Tabla 6.	Validación de requerimiento estado cuatro	55
Tabla 7.	Validación de requerimiento estado cinco	56
Tabla 8.	Validación de requerimiento estado seis	56
Tabla 9.	Validación de requerimiento estado siete.....	56

1. RESUMEN

En este trabajo se modeló una red de sensores inalámbricos (WSN, por sus siglas en inglés) con tres nodos que se comunican a través de Wifi, utilizando HiLeS (High Level Specification). Se desarrolló una implementación textual del metamodelo HiLeS, lo que permitió generar una representación visual del sistema. Esta solución facilita la simulación del comportamiento de cada nodo, además de permitir la obtención de una implementación concreta en MicroPython, lista para ejecutarse en plataformas ESP32.

2. INTRODUCCION

El acceso a la información es una característica fundamental en cualquier sistema. El auge de los sistemas de información ha impulsado cambios rápidos, apoyados por diversas tecnologías que permiten acceder a los datos según el contexto. El paradigma del Internet de las Cosas (IoT, por sus siglas en inglés) forma parte de esta nueva revolución que facilita el acceso a la información en tiempo real. En el ámbito de las Redes Inalámbricas de Sensores (WSN, por sus siglas en inglés), protocolos como LoRa, Zigbee, Bluetooth, GPRS o 3G integran un conjunto de tecnologías de comunicación que permiten diseñar WSN en diversos contextos. Factores como la tasa de transmisión, la cantidad de nodos, el tipo de información, la latencia, el ancho de banda, el consumo energético y la distancia entre nodos determinan la elección del protocolo y, por tanto, la arquitectura a emplear.

Por su parte HiLeS (Jiménez, 2000) es un lenguaje de descripción que permite de manera gráfica fijar funcionalidades jerárquicas de un sistema de estados y posteriormente validar la construcción de código ejecutable en sistemas de cómputo. Este lenguaje además ofrece un mecanismo para construir sistemas a la medida que cuenten con reglas de diseño, claras y ajustables a los requerimientos definidos.

El formalismo HiLeS permite definir reglas ajustables al contexto y debido a su naturaleza es posible construir una representación gráfica que posteriormente pueda entregar como

resultado código ejecutable sintetizado, trabajos como (G Muñoz, 2000) dan muestra de una herramienta que permite modelar y diseñar mediante especificaciones de alto nivel procesos continuos o discretos en forma práctica para posteriormente generar código ejecutable para algún sistema computacional, Por su parte (Jiménez, 2000) presenta modelos y herramientas para el diseño y síntesis de sistemas MEMS (MicroElectronics Mechanical Systems or Microsystems) mediante una descripción de alto nivel, allí define 3 aspectos necesarios para la implementación del desarrollo:

- Definición de Requerimientos.
- Diseño de Arquitectura.
- Implementación de la solución.

La validación estructural y el formalismo de alto nivel se realiza mediante el estándar de las redes de Petri. La definición del sistema y el diseño de la arquitectura se realiza mediante herramientas de Java. Luego de que el sistema esté definido y verificado se realiza una síntesis automática a módulos VHDL-AMS.

El paradigma IoT posibilita la toma de decisiones en cualquier ámbito en forma directa o indirecta. La implementación del paradigma en mega ciudades puede permitir que los costos y el consumo energético disminuyan, en (Gianni Pasolini, 2018) se puede ver como la ciudad de Montechiarugolo en Italia, ofrece servicio de parqueo, iluminación y control de tráfico inteligente a partir de adquisición de información telemétrica en postes de luz, entregando un ahorro energético del 76% en el periodo de un año, así mismo tecnologías como LoRa permiten la adquisición de datos entre coordenadas lejanas con bajo nivel de conectividad y el intercambio de medio para la transmisión de datos de acuerdo a circunstancias del medio. Por otro lado, investigaciones como (Nabeel Ahmed Malik, 2017) reflejan ahorros energéticos en infraestructuras IoT que utilicen el modo sleep como técnica de adquisición de datos y explican como este paradigma contribuye al desarrollo de la tecnología 5G en telefonías móviles al proveer la integración de información en contextos urbanos con lo cual es posible entregar beneficios en costos y eficiencia del espectro.

2.1 FORMULACION DEL PROBLEMA

El diseño de una WSN es un proceso complejo que involucra diversas variables y áreas de conocimiento. En este proceso intervienen disciplinas como la ingeniería de software, necesaria para desarrollar la aplicación que permita gestionar la información obtenida y hacerla accesible para el usuario; la arquitectura de hardware, responsable de la construcción de los dispositivos de adquisición de datos; los protocolos de enrutamiento y comunicación; y la seguridad de la información, esencial para garantizar la protección y alta disponibilidad de los datos, entre otros factores. Debido a la variedad de dispositivos y herramientas empleados en el diseño de una WSN, el uso de soluciones metodológicas puede facilitar su construcción de manera pragmática y funcional.

La arquitectura de una WSN depende de factores como el número de sensores, el tipo de comunicación (centralizada o distribuida), los protocolos de comunicación y enrutamiento, la ejecución en modo 'sleep', entre otros, que están ligados a las propiedades técnicas de los dispositivos utilizados. En este contexto, una descripción gráfica de la WSN que incluya estas variables permitirá hacer más práctica la construcción de una solución IoT.

El diseño de la solución se basa en un escenario general que abarca los conceptos fundamentales de una WSN, es decir, una relación de comunicación entre un coordinador y uno o más nodos, donde las propiedades de la red establecen las condiciones mínimas para su funcionamiento.

Para la descripción grafica se trabajara con una descripción de texto HiLeS ya que esta permite identificar estados y relacionarlos de manera secuencial. Se espera que de acuerdo con las funcionalidades del paradigma sea posible acoger las principales características de una WSN.

Pregunta de Investigación: ¿Cuál puede ser la metodología para implementar una red inalámbrica de sensores descrita mediante un modelo HiLeS ?

2.1.1 SISTEMATIZACION DEL PROBLEMA

¿Cuál es la representación textual que permita representar los bloques Hiles en lenguaje MicroPython?

¿Qué problema en el contexto de WSN y que conjunto de pruebas permitirán desarrollar la metodología propuesta?

2.2 OBJETIVOS

2.2.1 OBJETIVO GENERAL

Desarrollar una metodología para implementar un modelo HiLes de una WSN en lenguaje MicroPython.

2.2.2 OBJETIVOS EXPECIFICOS

- Encontrar una representación textual para los elementos HiLeS.
- Realizar una visualización de la representación textual de los elementos HiLes.
- Traducir la representación textual de los elementos HiLes en Lenguaje MicroPython.
- Definir un problema en el contexto WSN que permita probar la metodología propuesta mediante simulación lógica de una WSN.
- Desarrollar un conjunto de pruebas para validar el correcto funcionamiento de la WSN.
- Analizar los resultados obtenidos.

2.3 ACTIVIDADES A DESARROLLAR

Se plantean las siguientes actividades para llevar a cabo los objetivos propuestos:

1. Identificar el kit de desarrollo y las principales características de MicroPython.
 - a. Desarrollo de ejercicios prácticos y teóricos.
 - b. Validación Técnica.
2. Estudiar el paradigma Hiles, identificando sus elementos fundamentales.
 - a. Análisis del estado del arte.
 - b. Desarrollo de ejercicios prácticos.
3. Estructurar las características básicas para construir una arquitectura WSN.
 - a. Estado del arte.
 - b. Definición de variables básicas.
4. Implementar mediante Hiles una arquitectura WSN.

- a. Diagramar un nodo mediante un diagrama Hiles.
 - b. Diagramar el conjunto de nodos que pertenecen a la red.
5. Representar los elementos fundamentales del paradigma Hiles en lenguaje MicroPython
 - a. Obtener la representación en lenguaje MicroPython de un nodo diagramado en Hiles.
 - b. Obtener la representación en lenguaje MicroPython de algunos nodos en la red.
6. Representar la arquitectura del punto 4 en un dispositivo.
 - a. Implementar en dispositivos una arquitectura diagramada en HiLes usando MicroPython
 - b. Validar la ejecución y de acuerdo a los requerimientos funcionales.
7. Construir un escenario de pruebas que permita validar los resultados del diagrama HiLeS
 - a. Validar los resultados obtenidos, comparación teórica (resultado de estados diagrama Hiles) con resultados prácticos (resultado estados dispositivos).

3. MARCO TEORICO

3.1 RED INALAMBRICA DE SENSORES O WSN

Las redes inalámbricas de sensores o WSN, de acuerdo con lo enunciado por la Unión Internacional de Telecomunicaciones (ITU-T), se puede definir como “una red compuesta por nodos de sensores interconectados en forma inalámbrica que intercambia datos”, una WSN pueden ser clasificadas de diversas formas. En (Johan S., 2017) una WSN es clasificada como determinística, no determinística y móvil. En el despliegue determinístico las propiedades de red (IP's, formas de accesos, gateways, enrutamiento) está definido con antelación, los despliegues no determinísticos se encuentran en ambientes hostiles donde el hombre con dificultad puede intervenir, de esta forma, las propiedades de red deben ser dinámicas o auto reconfigurables evitando la dependencia del humano. Las redes

móviles se caracterizan debido a que los nodos de la red se desplazan por un perímetro predefinido. En otra clasificación (Srivastava, 2010) explica que las redes inalámbricas de sensores pueden ser de tipo estructurado y no estructurado. Las redes no estructuradas basan su funcionamiento en redes AD HOC, las cual no dependen de un control centralizado a diferencia de las redes estructuradas, que se caracterizan por tener un punto de acceso, coordinador o Gateway que permita el control de paquetes y comunicación con redes externas y nodos propios. Ahora bien. Con base en su implementación, área de trabajo y condición climática una WSN permite ser clasifica de la siguiente forma: WSN terrestre, WSN subterráneo, WSN sobre agua, WSN multimedia, WSN móviles. De acuerdo lo anterior cada tipo de red ofrecerá características en cuanto a consumo de energía, tasa de transferencia, reconfiguración, ancho de banda, procesamiento y compresión de datos, calidad de servicio y mantenimiento.

En (Nabeel Ahmed Malik, 2017) muestran como las tendencias en tecnologías eficientes en el ámbito de redes inalámbricas de sensores apuntan a disminuir el consumo de un nodo, aplicando técnicas en modo sleep y diseñando protocolos de enrutamiento que permitan adquirir la información de manera más rápida. El documento enuncia que se ha obtenido un ahorro del 19% de energía cuando la recolección de información se da entre nodos cercanos, en comparación de nodos lejanos. Así mismo menciona el concepto de red D2D (Dispositivo a Dispositivo) como una tecnología que concentra la transmisión de información entre nodos cercanos, con lo cual debería ser implementada en redes inalámbricas de sensores.

Para una WSN un **nodo o sensor** se puede definir como un dispositivo con la capacidad de capturar información de su medio y transmitirla. En este contexto, de acuerdo con (Srivastava, 2010) el desarrollo de microistemas electrónicos ha partido el avance en la construcción de sensores inteligentes, que además brinden propiedades computacionales y de almacenamiento. Actualmente los dispositivos de ultra bajo consumo son posibles gracias al desarrollo de chip radio de bajo consumo y algoritmos de bajo consumo que optimizan los tiempos transmisión.

3.1.1 PROTOCOLOS DE COMUNICACION

Una WSN en su implementación hace necesaria la definición de un protocolo de comunicación en ese contexto los protocolos a destacar son:

3.1.1.1 PROTOCOLO LORA

El protocolo LoRa (Alliance®, 2019) un protocolo de red de baja potencia diseñados para una extensión territorial amplia (LPWA), fue creada por un conjunto de organizaciones que se encargan de mejorar el protocolo y construir productos que trabajen con este. LoRa en general desarrolla una modulación inalámbrica que permite establecer redes de baja tasa de transferencia, bajo consumo y despliegue localizaciones territoriales amplias. En su especificación (LoRa Alliance, 2017) se identifican las siguientes capas:

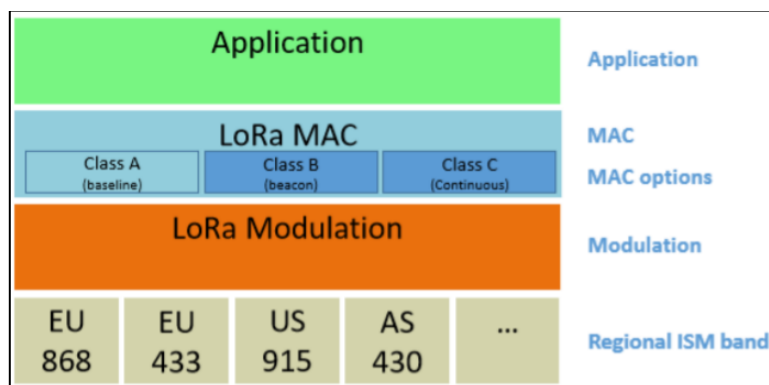


Figura 1. Capas protocolo LoRa

La capa inferior permite identificar la banda de frecuencia espectral en la cual trabajan, la clase de modulación es genérica para todos los dispositivos, por su parte la capa de control de acceso al medio (MAC) esta categorizada de acuerdo con el tipo de dispositivo clase A, B y C.

En (Gianni Pasolini, 2018) se desarrollan dos implementaciones del protocolo de comunicación Lora y dejan en evidencia sus características. En la primera implementación se construye una red inteligente de iluminación pública y en la segunda se implementa una WSN con largo alcance LWAN con el ánimo de acceder a mediciones medioambientales en varios puntos de un municipio. Las implementaciones muestran la ventaja del protocolo en este tipo de aplicación ya que permite modificar características en la red para permitir que la distancia de detección sea máxima, así mismo, identifica la necesidad de configurar un protocolo de comunicación multicanal que permita adquirir datos de diferentes receptores con un factor de expansión. El autor recomienda implementar un protocolo de comunicación machine - machine basado en NB-IoT.

3.1.1.2 PROTOCOLO ZIGBEE

El protocolo Zigbee (Alliance., 2020) un protocolo de red inalámbricas de sensores que tiene como base el estándar IEEE 802.15.4, En general el protocolo cuenta con las siguientes características:

- Permite trabajar en la banda de 2.4GHz con múltiple portadora.
- Permite trabajar en la banda de 915Mhz – 868Mhz.
- Incorpora mecanismos para disminuir el consumo de energía para todos los dispositivos que soportan el protocolo.
- Ofrece técnicas de seguridad basadas en claves, con lo cual se pueden generar métodos de emparejamiento.
- Permite la implementación de redes de área personal (PAN) mediante topología estrella.
- Ofrece alternativas de transmisión como broadcast, groupcast y unicast.
- Permite gestión de potencia, es decir que los dispositivos se comuniquen de acuerdo a un nivel establecido.
- La transmisión en los canales por debajo de GHz varía hasta 1 km.

La nueva versión 3.0 ofrece productos avalados por estándares certificados que avalan la puesta en producción del mismo, las capas de un producto soportado por el estándar tienen las siguientes características:

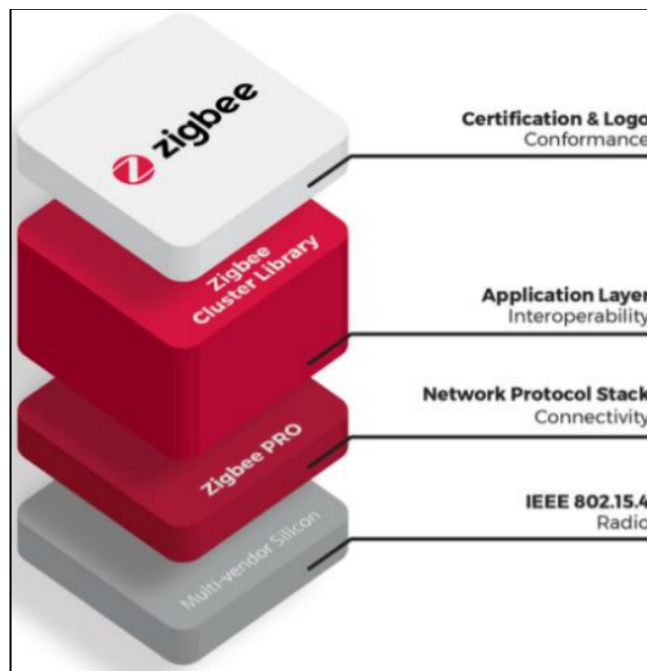


Figura 2. Capas protocolo Zigbee

3.1.1.3 PROTOCOLO WIFI

Soportado bajo el estándar IEEE 802, el protocolo wifi es el más implementado en redes inalámbricas, cuenta con alta cantidad de versiones, que mejoran desde aspectos de seguridad hasta el control de acceso al medio. Organizaciones como wifi Alliance al igual que en los anteriores protocolos, fabrican y soportan la implementación de productos en comunicación wifi. De acuerdo con (J Garbarino, 2011) el protocolo ofrece:

- Altas tasas de transmisión y medidas de QoS (Calidad de servicio).
- Enrutamiento de salto único para redes ad-hoc.
- Alta potencia de transmisión.

Aunque el protocolo no está diseñado exclusivamente para redes inalámbricas de sensores. Su especificación permite adaptarla, y de esta manera aprovechar sus ventajas.

3.1.1.4 PROTOCOLO SIGFOX

El protocolo Sigfox (Community, 2019) es un servicio orientado a dispositivos IoT, diseñado para vincular cualquier dispositivo al internet. La vinculación de los dispositivos directamente con la nube permite realizar una adecuada gestión de energía en el dispositivo. Esta tecnología es compatible con los protocolos Bluetooth, GPS 2G/3G/4G y Wifi. Como características principales se destacan:

- Transmisión asíncrona entre dispositivo y nube.
- Parámetros configurables a disposición. (Por ejemplo, el dispositivo puede enviar mensajes broadcast 3 veces por distinto canal y frecuencia), esto es conocido como Frecuencia Hopping.
- Uso de radio cognitiva para identificar alto tráfico y evitar el uso de canales saturados.
- Baja tasa de transmisión y alto rango de cubrimiento y calidad de servicio.

3.2 FORMALISMO HILES

- El formalismo HiLeS, (C. E. Gómez, 2010) lo define como una especificación de alto nivel que permite modelar sistemas continuos o discretos a partir estados para posteriormente mediante redes de Petri realizar la validación del sistema. En (C. E. Gómez, 2010) por ejemplo, HiLeS es usado para diseñar un convertidor tipo “interleaved boost”, allí se utiliza el concepto de redes de Petri para analizar la estabilidad del sistema y mediante herramientas como Hiles-Designer (LAAS-CNRS) desarrollar el código VHDL-AMS. Los elementos que componen la descripción son:
- **Bloques estructurales:** Son representados por rectángulos y permiten la definición jerárquica del sistema, cada bloque interactúa con canales de entrada y salida para recibir y entregar información. Un bloque estructural puede contener bloques funcionales dentro de él.



Figura 3. Bloque estructural – HiLeS

Un bloque estructural puede representar un sensor de humedad que entrega una señal con el valor de humedad y una señal como tasa de transmisión.

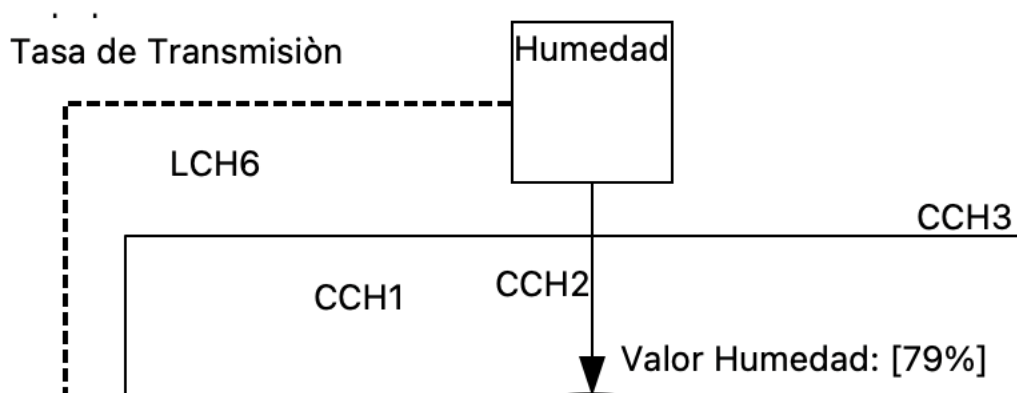


Figura 4. Bloque estructural sensor de humedad

Así mismo un bloque estructural puede representar un nodo en una arquitectura de red inalámbrica de sensores, que reciba señales de otros nodos de la red o envíe señales a otros nodos. En la siguiente figura se define un nodo de visualización con un bloque estructural que recibe una señal de riego y envía un señal de solicitud de status.

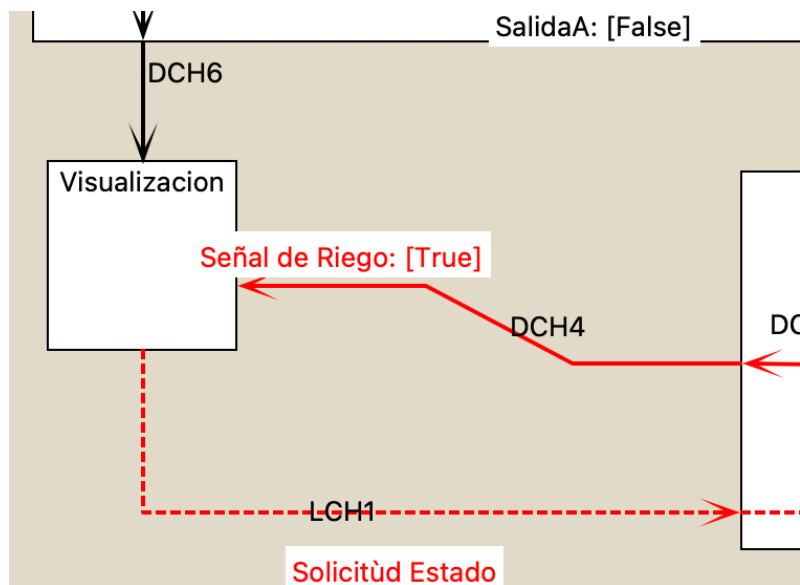


Figura 5. Bloque estructural que representa un nodo en una WSN

- **Bloques Funcional:** Son representados por rectángulos de esquina redonda y contienen la descripción del sistema mediante ecuaciones diferenciales, algebraicas o lógicas. Un bloque funcional no puede contener un bloque jerárquico.



Figura 6. Bloque funcional – HiLeS

En la siguiente figura, el bloque funcional controla el valor de humedad entregando un valor de true en CCH4 si el valor de humedad es mayor a 79%.

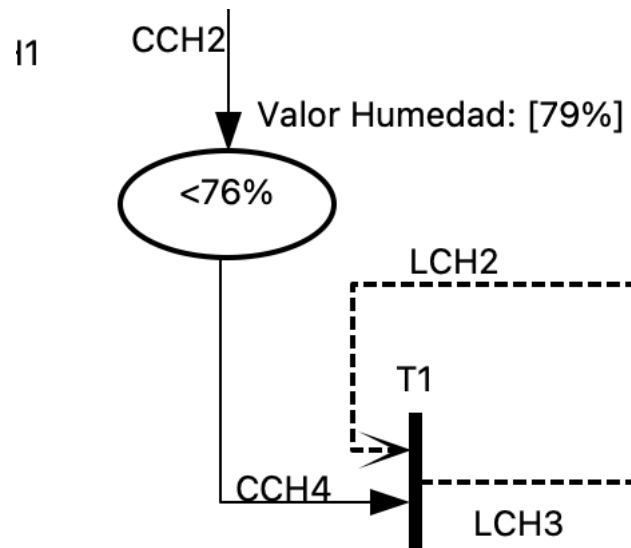


Figura 7. Bloque funcional para control de humedad mayor a 76%

- **Los canales** en una descripción HiLeS, describen el tipo de señal que interactúa con un bloque, existen tres tipos, continuo, discreto o Red de Petri. Cada canal cuenta con su notación particular.



Figura 8. Canal continuo



Figura 9. Canal evento discreto



Figura 10. Canal Red de Petri

El canal de red Petri describe una señal de control que permite definir estados lógicos, el canal continuo/discreto transporta información inherente al sistema (Señal eléctrica, física, electromagnética, térmica, entre otros).

- **Los servicios** son puntos de acceso y vinculación para intercambio de información con el sistema, por otro lado, **los puertos** representan intercambio de información entre bloques y canales internos del sistema.

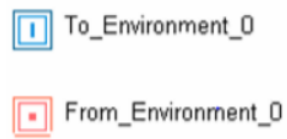


Figura 11. Servicios HiLeS



Figura 12. Puertos - HiLeS

Utilizando los elementos descritos, HiLes permite describir uno o múltiples niveles jerárquicos para el sistema. Por ejemplo:

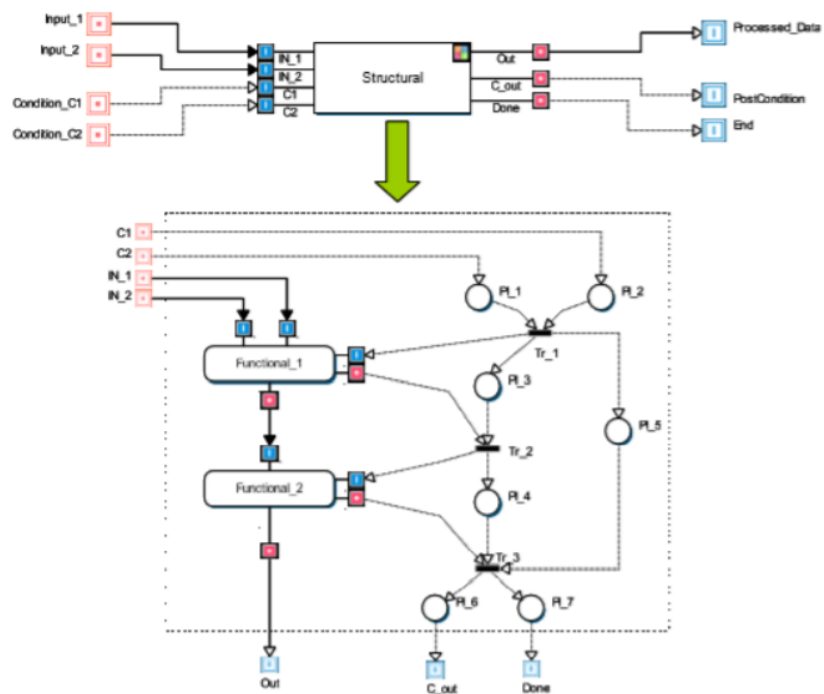


Figura 13.Nivel jerárquico uno

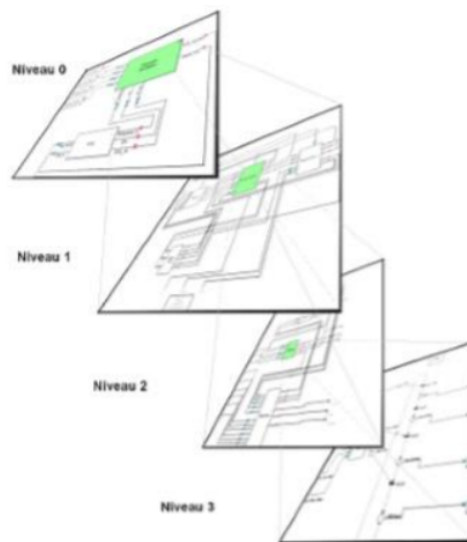


Figura 14. Múltiples capas jerárquicas

- **Sample y Hold:** En (Jiménez, 2000) se evidencia la descripción de nuevo componentes que permiten convertir un tipo de señal mediante una operación de “sampling”. (Jiménez, 2000) elabora una notación para construir una representación de conversión de señal continua a discreta acciona por una señal de control.

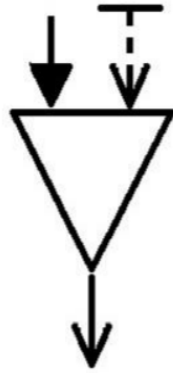


Figura 15. Sample and Holding A

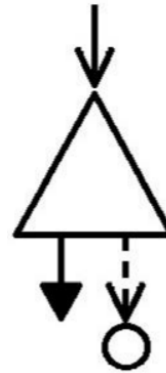


Figura 16. Sample and Holding B

La figura A (Sample and Holding A) detalla el ingreso de una señal continua que conservará su estado y será entregada como señal discreta al ingreso de una señal lógica. De la misma forma la figura B (Sample and Holding B) representa la notación de una conversión de señal discreta en señal continua, en este caso la señal de control se produce cuando la señal discreta se presente, por su parte el estado de la señal continua se conservará a lo largo del tiempo.

Un elemento Sample resulta útil para representar la invocación de una señal en un nodo de una WSN.

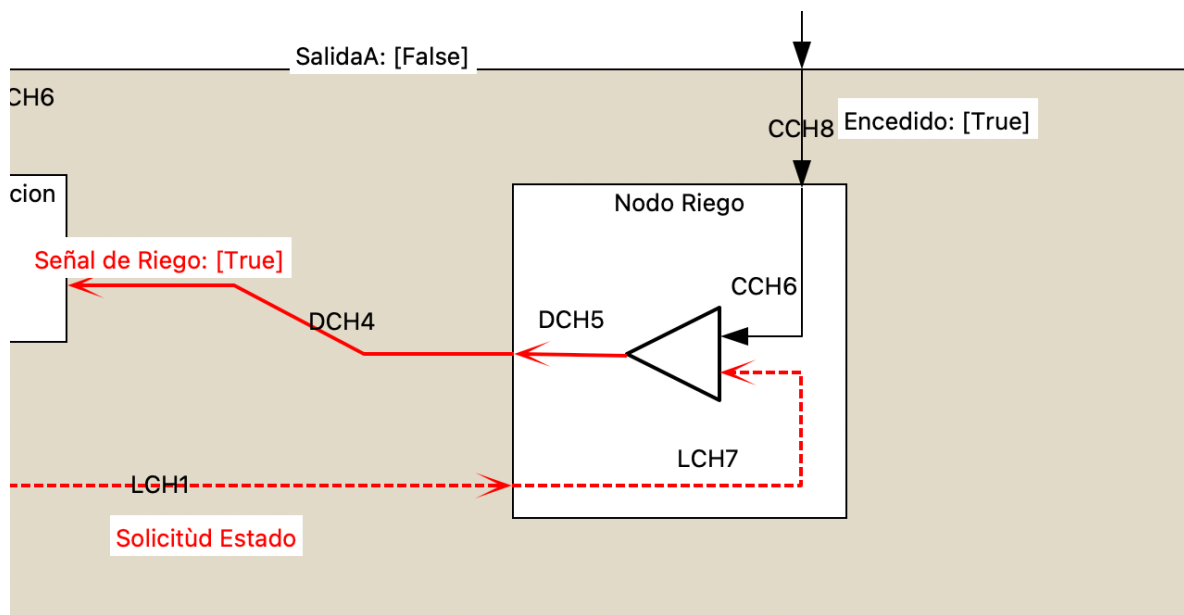


Figura 17. Uso de operador Sample en un WSN

En la anterior figura la operación Sample permite definir el comportamiento que tiene la señal de riego en un nodo, en este caso, cada vez que llegue una solicitud de status por el canal LCH7, el valor de encendido o apagado que tenga el canal CCH6 (proveniente de otro nodo) se emite por el canal DCH5 hacia el nodo destino.

La red de Petri : Aunque no hace parte de la descripción formal, permite reproducir comportamientos de un sistema a partir de eventos discretos, se compone principalmente de:

- **Transiciones:** Elemento que permite controlar el paso de un token de acuerdo al estado una señal, necesita de N entradas lógicas con valor de true/false, una entrada de token y una salida de token.

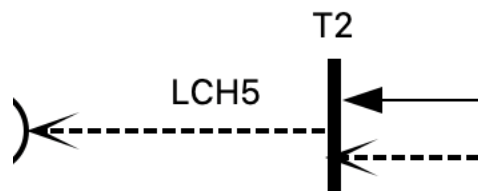


Figura 18. Representación de una transición en una red de Petri

- **Lugares:** Elemento que almacena un token.

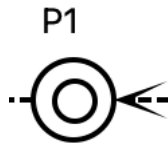


Figura 19. Representación del elemento lugar en una red de Petri

- **Token:** Elemento que ocupa un lugar.

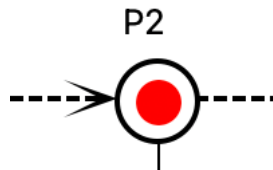


Figura 20. Representación de un token en una red de Petri

A manera de ejemplo, la siguiente imagen modela la señal de encendido/apagado de un nodo de riego en una WSN .

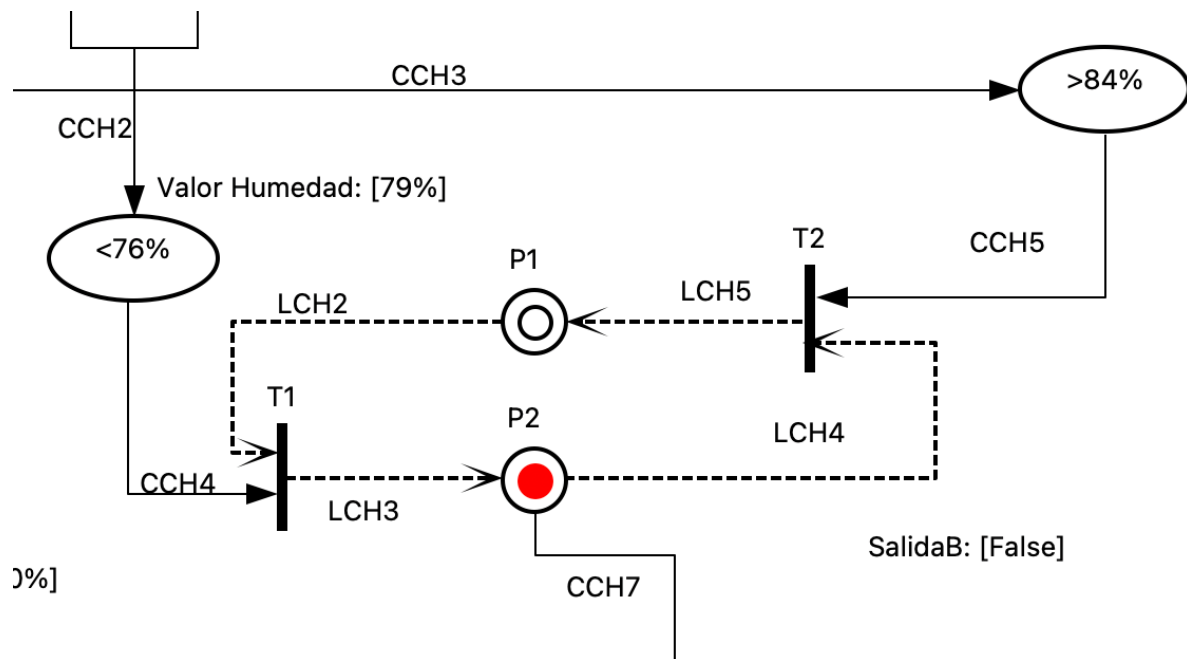


Figura 21. Red de Petri para encendido y apagado para un nodo de riego.

El token en P2 representa el encendido, una vez CCH5 sea true, lo cual ocurre cuando CCH3 sea mayor a 84%, el token cambiara al lugar de P2 a P1, lo cual representa que el nodo de riego se apagara, de la misma forma si el token se encuentra en P1 una vez la señal CCH2 sea menor a 76% el token pasara de P1 a P2 por que CCH4 es true.

3.3 DISPOSITIVO SYSTEM ON CHIP

Los **dispositivos System on Chip** (Greaves, 2016) son circuitos compactos que detallan conjuntos de subsistemas como CPU, memorias, interfaces I/O, Timers, interfaces de radio, GPIO, entre otros, de tal forma que permiten construir soluciones particulares dirigidas a una WSN. A continuación, se describe la arquitectura general de los dispositivos SoC.

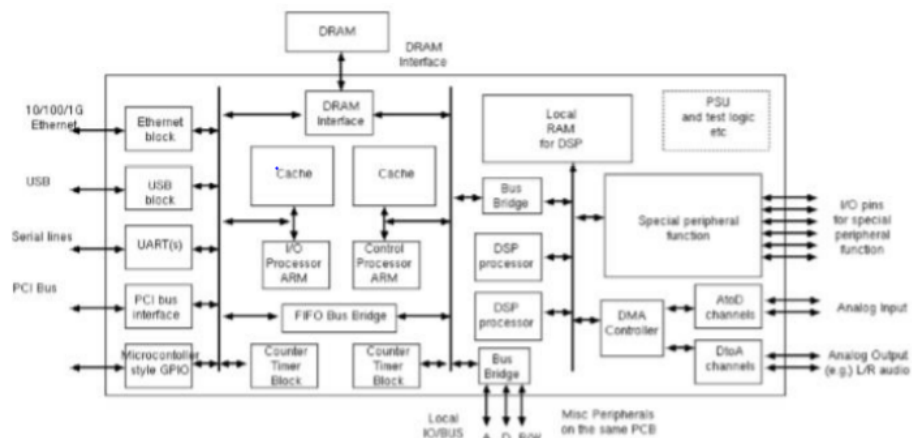


Figura 22.Arquitectura SoC

La tecnología permite definir los componentes necesarios como: un sistema de comunicación inalámbrica acogida en un protocolo de comunicación, modelar reglas de muestreo de la información, emular comunicaciones cliente servidor asíncronas o síncronas, entre otras características detalladas por una WSN. Así mismo en el mercado se encuentran soluciones dirigidas al contexto IoT que permiten implementar soluciones para problemas como: Consumo de energía, protocolos de comunicación especiales, tamaño de dispositivo, entre otras.

4. ACTIVIDADES DESARROLLADAS

4.1 PLANTEAMIENTO DE WSN Y MODELAMIENTO FORMAL

A manera de ejemplo se plantea el siguiente problema:

Controlar las variables medio ambientales de una planta, con el fin de mejorar su ciclo de vida, la planta necesita de calefacción para mantener su temperatura entre 12C° - 19C°, además de un índice de humedad de 80% y una PH 0.14.

Elementos que conforman la red:

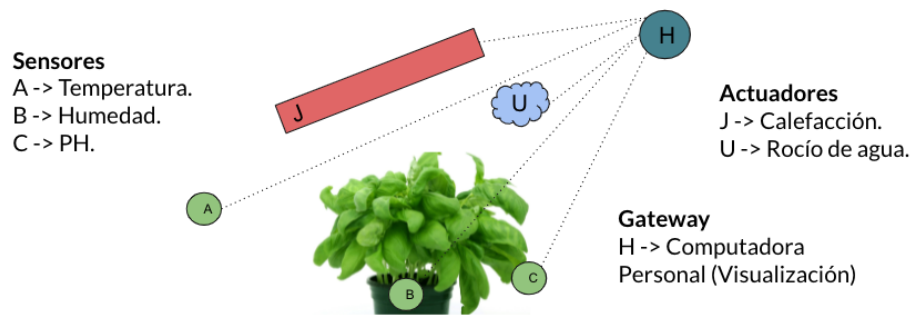


Figura 23.Red Inalámbrica de sensores para planta

El sistema cuenta con un nodo para medir la temperatura, un nodo para medir la humedad, un nodo para medir el PH, un nodo de riego, un nodo de calefacción y un nodo para visualización. Los dispositivos tienen una implementación de código en MicroPython, la red podrá es centralizada y usa el protocolo wifi como mecanismo de comunicación.

El modelamiento formal del sistema es:

- **Diagrama de requerimientos:**

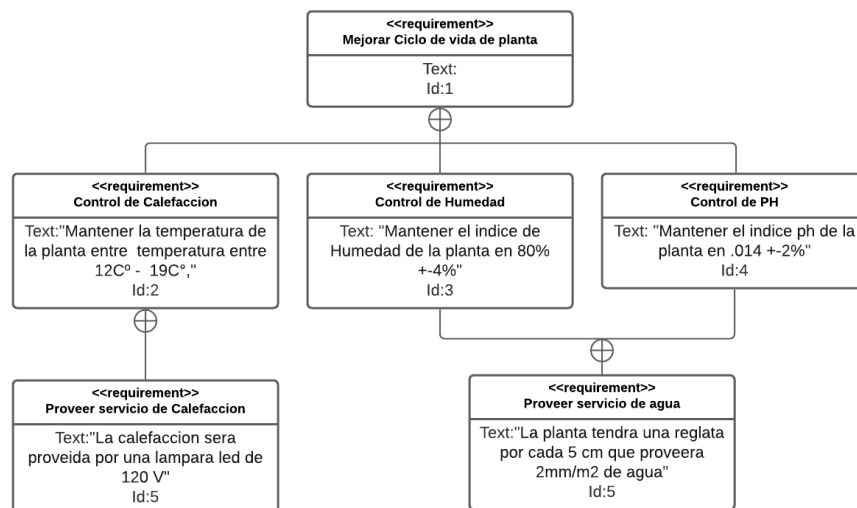


Figura 24.Diagrama de requerimientos ciclo de vida planta

- **Diagrama definición de bloques:**

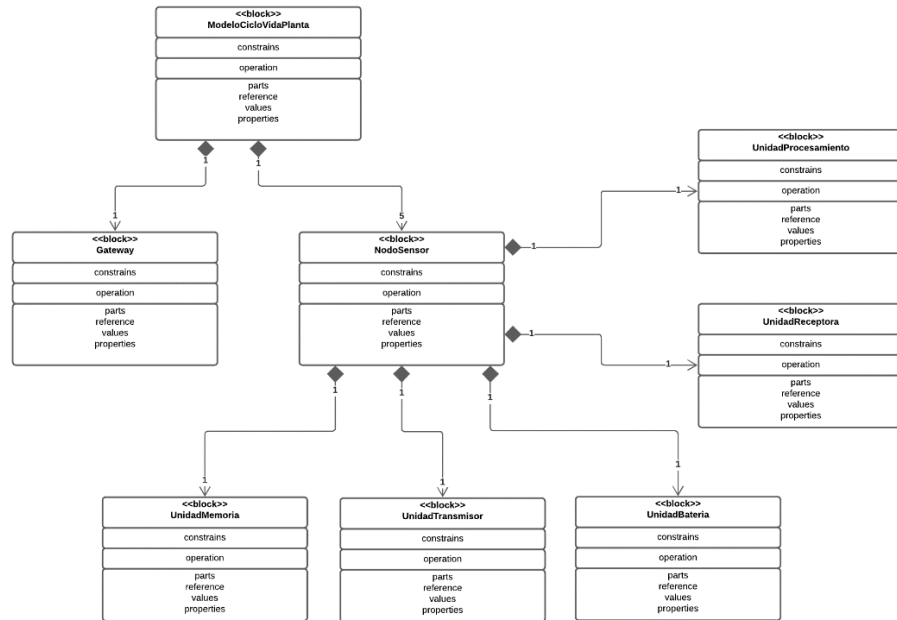


Figura 25. Diagrama definición de bloques

- Diagrama Paramétrico:**

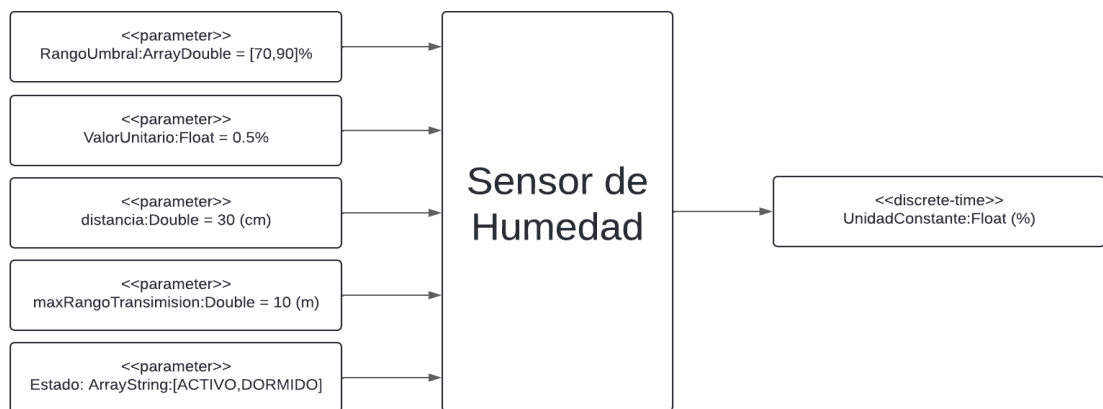


Figura 26. Diagrama paramétrico sensor de humedad

- **Diagrama máquina de estados Transmisor:**

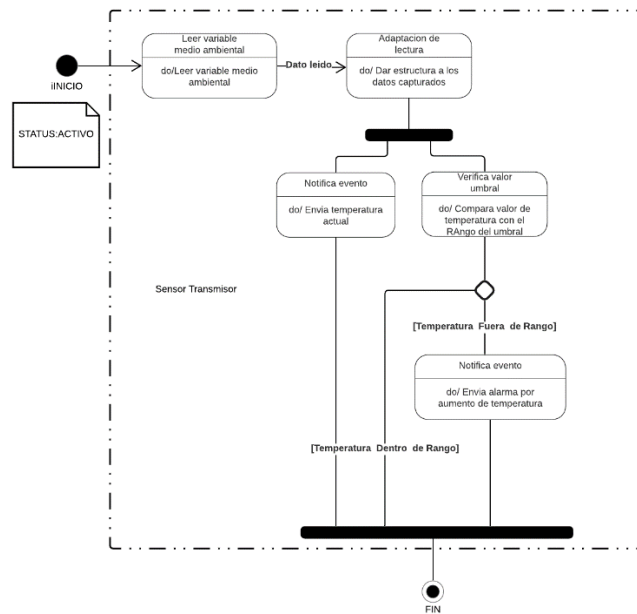


Figura 27. Diagrama máquina de estados Transmisor

- **Diagrama máquina de estados Receptor:**

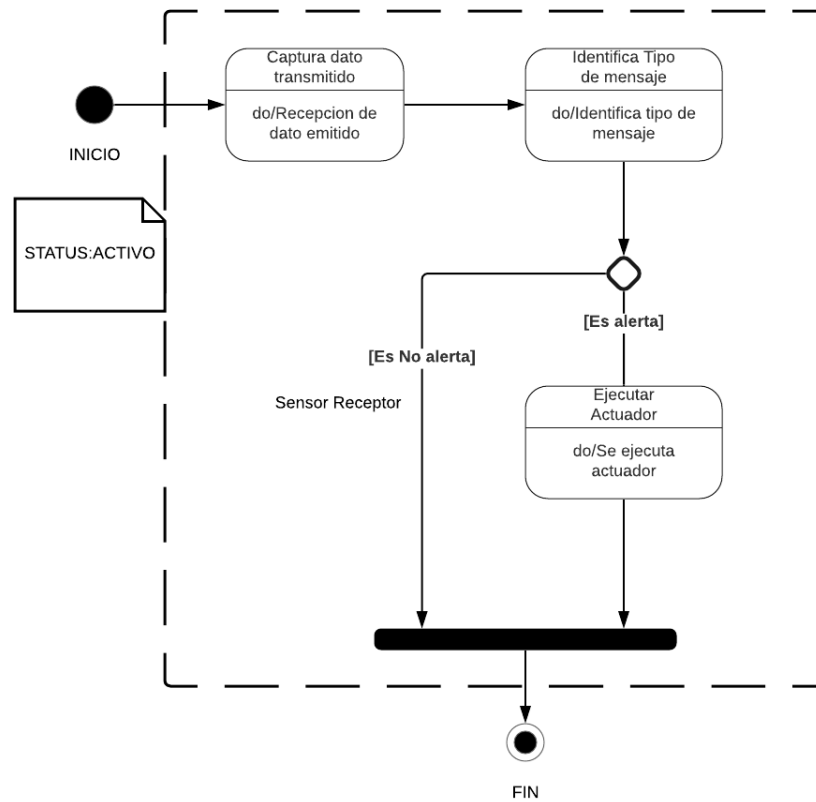


Figura 28.Diagrama máquina de estados Receptor

El diagrama HiLes del para el nodo de humedad , riego y visualización se define como:

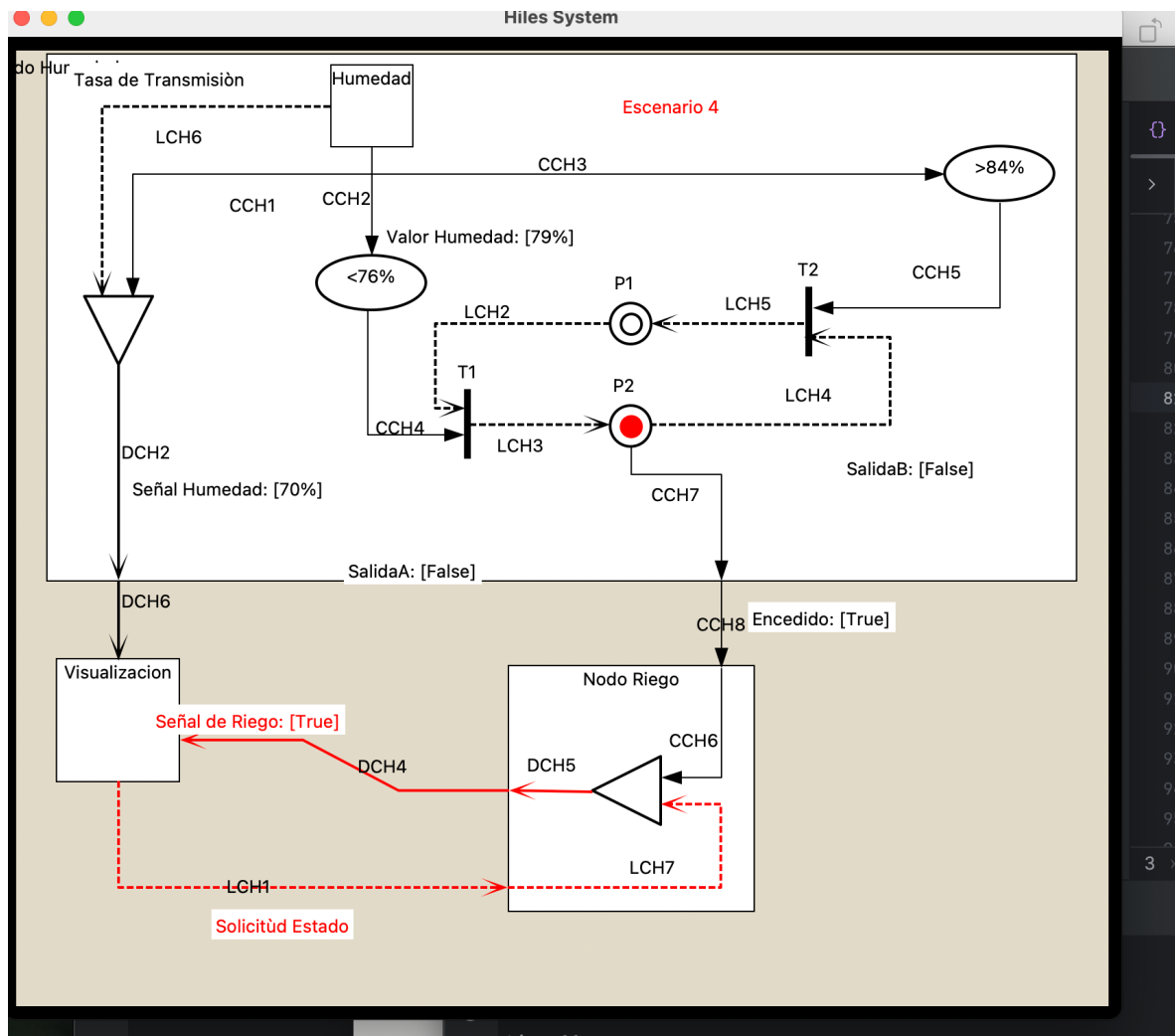


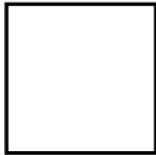
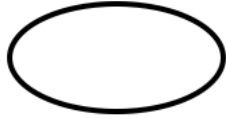
Figura 29. Implementación HiLeS Red Inalámbrica ejemplo

Cada nodo hace uso de un bloque estructural para su representación, el nodo de visualización se asume como una caja "negra" en donde se desconoce de qué forma se genera la solicitud de estado y como se visualizan las señales de humedad y de riego, por su parte el nodo de humedad hace uso de una red de Petri para determinar en qué momento enviar la señal de encendido hacia el nodo de riego, el operador Sample permite en el nodo de riego modelar el comportamiento de transmisión hacia el nodo de visualización. Por su parte el operador Sample del nodo de humedad permite representar la señal emitida hacia el nodo de visualización, cada vez que el sensor de humedad emita un 1 en el canal LCH6, el valor de humedad será emitido al nodo de visualización.

- El objeto “ElementHiLeS” será el encargado de agrupar los elementos HiLeS que serán utilizados (Bloque funcional, Bloque estructural, Puertos, Servicios, Puertos, Sample, Hold, lugares y transiciones)
- Un conector vinculara dos elementos HiLeS.
- La red de Petri está compuesta por lugares y las transiciones.
- Se aseguran algunas reglas de la especificación HiLeS, como lo son:
 - Las transiciones aceptan conectores del tipo discreto o continuo.
 - Los servicios son usados por los bloques estructurales y los puertos por los bloques funcionales.
 - Una arquitectura está compuesta de varios elementos HiLeS.
- La implementación de una capa de abstracción que permitirá realizar una representación visual.

4.3 REPRESENTACION VISUAL DE LOS ELEMENTOS HILES

Para la representación visual se elige trabajar con la librería de tkinter y realizar una implementación que permita graficar los elementos HiLeS. Como decisión de diseño se construye un objeto que permitirá graficar los diferentes elementos HiLeS sin alterar la definición propuesta en el ítem anterior. De acuerdo a esto:

Representación	
Textual	Visual
<pre>structuralBlock = architecture.structuralBlock(name ='structuralBlock 1', centerX=250,centerY=250,length=400)</pre>	
<pre>funtionalBlock =architecture.funtionalBlock(name="functionalBlock 1", relativePointX=10,relativePointY=10,length=400)</pre>	
<pre>service = architecture.service(structuralBlock,name='service 1', centerX=10,centerY=10,length=12,type="OUT")</pre>	

<pre>port = architecture.port(functionalBlock=functionalBlock,name="port 1", centerX=90,centerY=90,length=12,type="OUT")</pre>	
<pre>discreteChannel = architecture.discreteChannel(name='channel d1', from_ =service, to_=port, initialPointX=400,initialPointY=400, finalPointX=300,finalPointY=300)</pre>	
<pre>continuousChannel = architecture.continuousChannel(name='channel c1', from_ = service,to_=port, initialPointX=410,initialPointY=400, finalPointX=310,finalPointY=300)</pre>	
<pre>logicalChannel = architecture.logicalChannel(name='channel l1', from_ = service,to_=port, initialPointX=420,initialPointY=400, finalPointX=320,finalPointY=300)</pre>	
<pre>place = netPetri.place(name='place 1',hasToken=True, relativePointX=80,relativePointY=65,length=15)</pre>	
<pre>transition = netPetri.transition(name = 'transition 1', centerX=50,centerY=50,length=20)</pre>	
<pre>sample = architecture.sample(name='sample 1',direction='LEFT', relativePointX=150,relativePointY=350,length=20)</pre>	
<pre>hold = architecture.hold(name='hold 1',direction='UP', relativePointX=250,relativePointY=190,length=20)</pre>	

Tabla 1.Representación Textual y visual de los elementos HiLeS

La representación entrega el resumen de su implementación resaltando en rojo algunas asociaciones no permitidas, por ejemplo:

```
oscarjavieridrobo@BOMC8092016 ProyectoHiles % python3 HilesSystem.py
Element created: structuralBlock 1
Element created: functionalBlock 1
Element created: service 1
Element created: port 1
Connecting service 1 to port 1
The discrete connection channel d1 was True
Connecting service 1 to port 1
The continuous connection channel c1 was True
Connecting service 1 to port 1
The logical connection channel l1 was True
Element created: place 1
Element created: transition 1
Element created: sample 1
The connector channel d1 not was assigned to Sample output
The connector channel l1 was assigned to Sample output logical
Element created: hold 1
The connector channel d1 not was assigned to Hold input
The connector channel l1 was assigned to Hold input logical
```

Figura 31.Resultado descripción textual de sistema HiLeS

4.4 EVALUACION DE LOS DISPOSITIVOS DE IMPLEMENTACION

Las características para tener en cuenta para la evaluación son: Estándar de comunicación, Consumo, capacidad de cómputo, entorno de desarrollo y costo.

4.4.1 ESP8266

Es un chip de bajo costo fabricado por Expressif, soporta todo el stack TCP/IP,UDP,HTTP y FTP, bajo el estándar de comunicación WIFI, su fabricante destaca las siguientes características:

- **Alta durabilidad:** trabaja en amplio rango de temperatura, ofreciendo un chip rentable, compacto y robusto.
- **Arquitectura que ahorra energía:** diseñado especialmente para dispositivos móviles y aplicaciones IoT, cuenta con estado de bajo consumo y tres modos de operación, activo, dormido y dormido profundo.

- **Compacto:** Procesador de 32 bits (Tensilica L106 32-bit RISC - 160 MHz), interfaces digitales, interfaz de radio wifi, amplificador de poder, amplificador receptor de bajo ruido, filtros y módulos para la gestión de potencia.

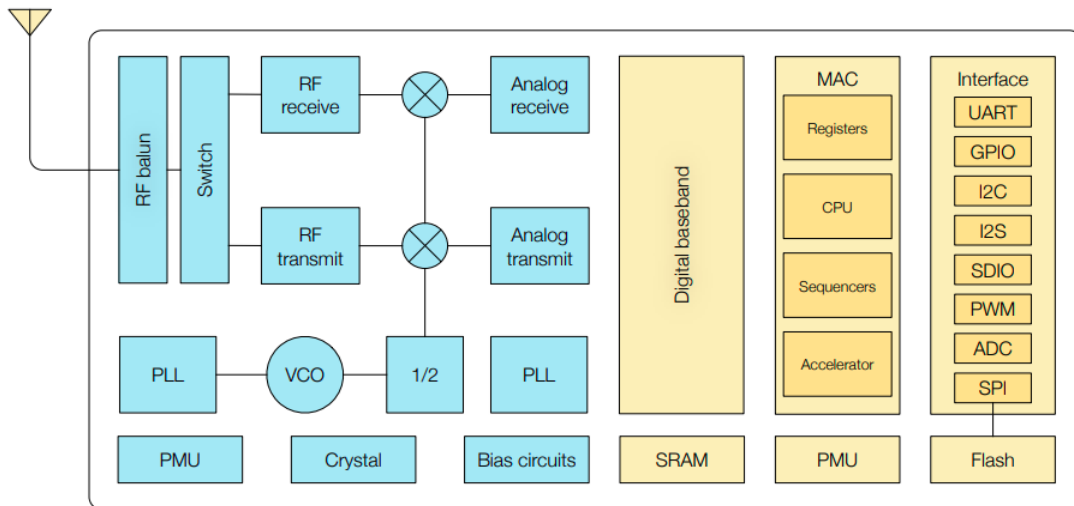


Figura 32. Diagrama de bloques ESP8266EX

Categories	Items	Parameters
Wi-Fi	Standard	FCC/CE/TELEC/SRRC
	Protocols	802.11 b/g/n/e/i
	Frequency Range	2.4 G ~ 2.5 G (2400 M ~ 2483.5 M)
	Tx power	802.11 b: + 20 dBm
		802.11 g: + 17 dBm
		802.11 n: + 14 dBm
	Rx Sensitivity	802.11 b: -91 dBm (11 Mbps)
		802.11 g: -75 dBm (54 Mbps)
		802.11 n: -72 dBm (MCS7)
Hardware	Antenna	on-board, external, IPEX connector, ceramic chip
	Peripheral interface	UART/SDIO/SPI/I2C/I2S/IR Remote Control
		GPIO/PWM
	Operating voltage	2.5 V ~ 3.6 V
	Operating current	Average: 80 mA
	Operating temperature range	-40 °C ~ 125 °C
	Storage temperature range	-40 °C ~ 125 °C

Figura 33.Especificaciones Técnicas ESP8266EX Parte 1

Categories	Items	Parameters
	Package size	QFN32-pin (5 mm x 5 mm)
	External interface	N/A
Software	Wi-Fi mode	Station/SoftAP/SoftAP+Station
	Security	WPA/WPA2
	Encryption	WEP/TKIP/AES
	Firmware upgrade	UART Download/OTA (via network)
	Software development	SDK for customized development/cloud server development
	Network Protocols	IPv4, TCP/UDP/HTTP/FTP
	User configuration	AT Instruction Set, Cloud Server, Android/ iOS app

Figura 34. Especificaciones Técnicas ESP8266EX Parte 2

Este dispositivo tiene algunos similares dentro de los cuales se destacan:

- ESP8285: Ofrece 1MB de memoria flash extra.

Dentro de las principales plataformas de desarrollo se destaca:

- NodeMCU - Lenguaje Lua
- Wemos D1

Lenguajes, librerías o implementaciones firmware

Core for Arduino IDE - Implementación Arduino.

MicroPython - Librería basada en python.

En cuanto kit de desarrollo tenemos la placa

- [ESP-Launcher](#)
- [ESP8266-DevKitC](#)

Costo: \$15.000 (NodeMcu)

4.4.2 ESP32

Es una versión mejorada de SoC que adiciona comunicación Bluetooth de modo dual integrada, además de extender su procesador a dos núcleos, e incluir interruptores de antena, su fabricante destaca las siguientes características:

- **Diseño Robusto:** Cuenta con un rango de trabajo entre -40°C y 125°C, lo cual es posible debido a circuitos de calibración avanzados que corrigen las imperfecciones provocadas por el ambiente de trabajo.
- **Consumo de energía ultra bajo:** Incluye varios modos de energía, escalado dinámico y sincronización fina de reloj.

- **Alto nivel de Integración:** Debido que permite intercambiar la antena y cuenta con balun de RF, amplificadores de poder, receptores amplificador de bajo ruido, y módulos para gestión de energía, todo lo anterior con mínimos requisitos en su PCB

Protocolo de comunicación Wifi/Bluetooth : Puede proveer la funcionalidad de interfaz wifi y bluetooth a través de las integraciones mediante SPI/SDIO/I2C y UART.

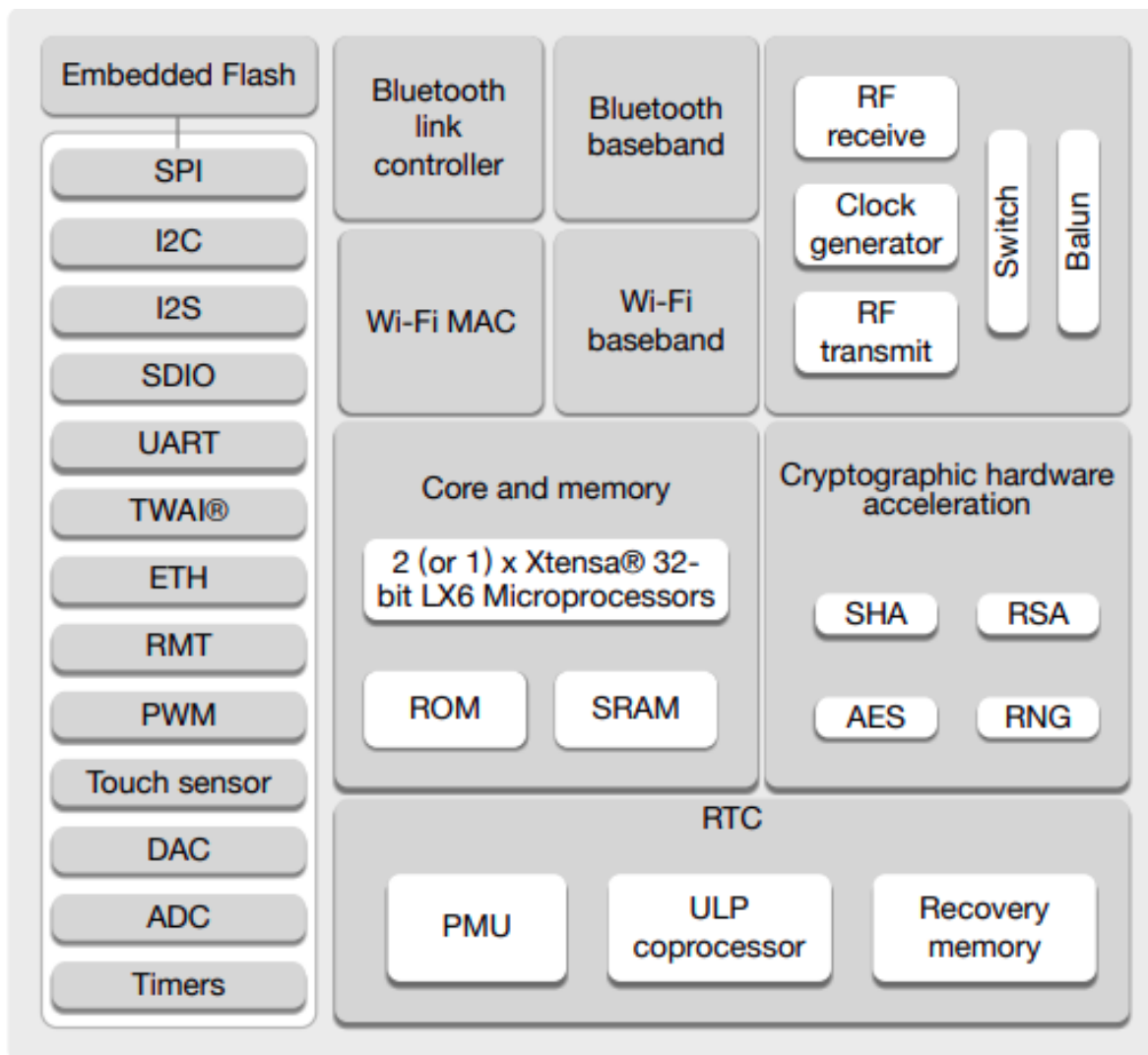


Figura 35. Diagrama de bloque funcional ESP32

Características	Ítem	Parámetro
-----------------	------	-----------

CPU y Memoria	Frecuencia de Trabajo	240 MHz
	Procesador	one or two low-power Xtensa® 32-bit LX6 m
	Soporte	Instrucciones DSP, interrupciones, Unidad de punto flotante.
	Memoria interna	448 KB de ROM y 520 KB de SRAM
	Memoria Flash	16 MB
Timers y Watchdogs	Características	reloj de 16 bits, Timer de 64 bits, configuración up/down.
	RTC Clock	Oscilador Interno de 8MHz, reloj interno de 31.25 KHz, reloj PLL
Wi-Fi	Protocolo	Staff 802.11
	Potencia transmisión	20.5 dBm
	Integración de antena	Si
	Interfaces virtuales	4
Bluetooth	Potencia transmisión	21 dBm
	Protocolo	Bluetooth v4.2 BR/EDR and Bluetooth LE
Modo de operación	Tipos	Modo activo, Modo dormido, Dormido liviano, Modo dormido profundo, Modo Hibernación
GPIO	Cantidad	34
Conversores	ADC	12-bit SAR
	DAC	2 de 8 bits
Sensores	Efecto Hall	respuesta vinculada al ADC
	Contacto	Del tipo capacitivo
Interfaces	UART	Cantidad 3
	I2C	Modo standard (100 Kbit/s)

		Modo rápido (400 Kbit/s)
	I2S	

Tabla 2.Especificación Técnicas ESP32

Este dispositivo tiene algunos similares dentro de los cuales se destacan:

ESP32-S3: Ofrece conectividad con Bluetooth 5 (LE).

ESP32-S2: Diseñado para ser seguro y rentable, aumenta sus funcionalidades en términos de seguridad y su capacidad en I/O.

ESP32-C3: Diseñado con una MCU RISC-V rentable y conectividad Bluetooth 5 (LE) para aplicaciones seguras de IoT

Lenguajes, librerías o implementaciones firmware:

Espressif IoT Development Framework

En cuanto kit de desarrollo tenemos la placas:

- ESP32-DevKitC
- ESP32-DevKitM-1
- ESP-WROVER-KIT
- ESP32-PICO-KIT
- ESP32-PICO-DevKitM-2

Costo: \$36.000 (Kit de desarrollo)

4.4.3 PRODUCTOS DIGI XBEE

Es un ecosistema tecnológico que soporta el protocolo de comunicación ZigBee y GPS y Bluetooth, estos productos cuentan con diversas soluciones modulares que pueden hacer parte de una red inalámbrica de sensores. Se destaca el módulo Digi XBee PRO S2C Zigbee debido al costo/beneficio y accesibilidad al mercado.

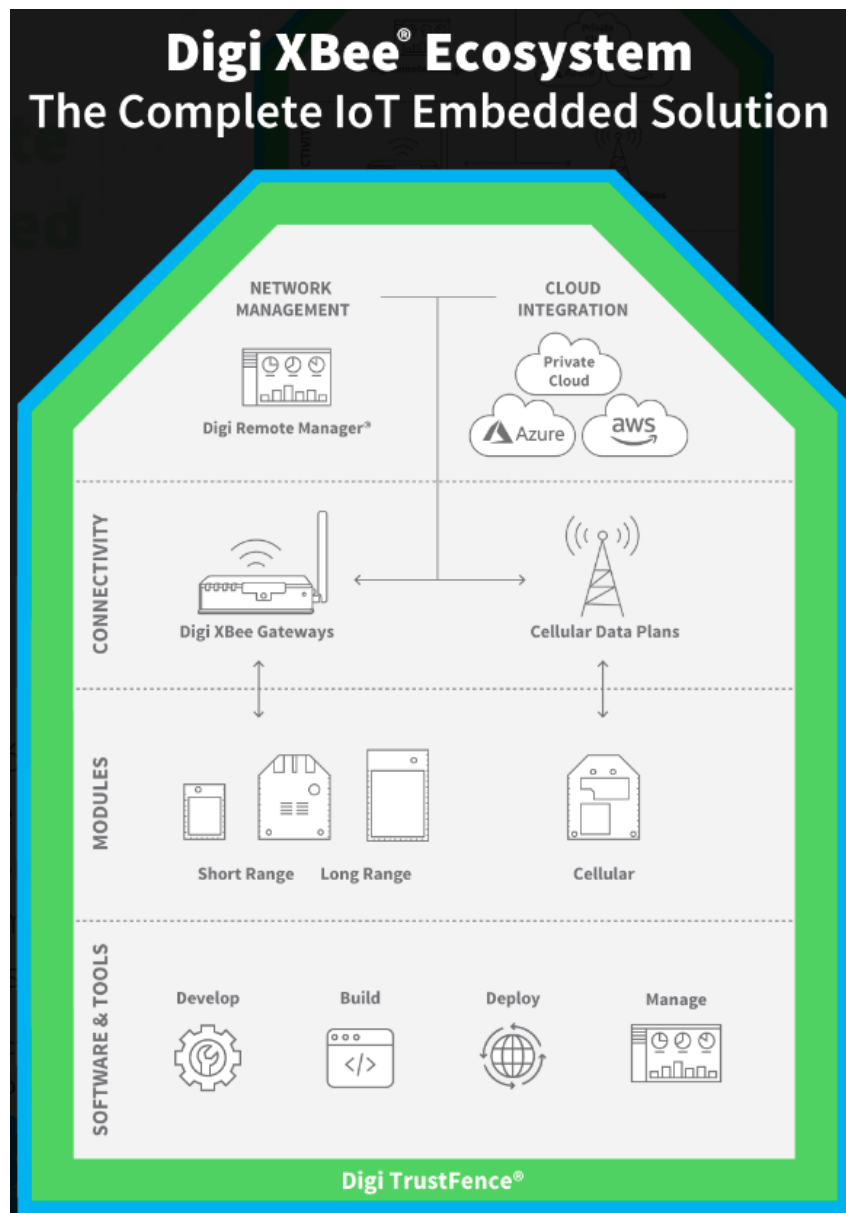


Figura 36. Esquema Digi XBee

Las características básicas de esta versión son:

- Permite implementar aplicaciones Zigbee que sean de estado crítico.
- Implementación en modo sleep. (consumo de 1uA)

- Incorpora la comunicación SPI (serial peripheral interface) que provee un intercambio de datos con una alta velocidad entre dispositivos.

En cuanto a especificaciones técnicas tenemos lo siguiente:

SPECIFICATIONS		Digi XBee® S2C Zigbee	
		Standard	Programmable
PERFORMANCE			
TRANSCIEVER CHIPSET	Silicon Labs EM357 SoC		
DATA RATE	RF 250 Kbps, serial up to 1 Mbps		
INDOOR/URBAN RANGE*	Up to 60 m (200 ft)		
OUTDOOR/RF LINE-OF-SIGHT RANGE*	Up to 1200 m (4000 ft)		
TRANSMIT POWER	3.1 mW (+5 dBm) / 6.3 mW (+8 dBm) boost mode		
RECEIVER SENSITIVITY (1% PER)	-100 dBm / -102 dBm boost mode		
FEATURES			
SERIAL DATA INTERFACE	UART, SPI		
CONFIGURATION METHOD	API or AT commands, local or over-the-air (OTA)		
FREQUENCY BAND	ISM 2.4 GHz		
FORM FACTOR	Through-hole, surface mount		
INTERFERENCE IMMUNITY	DSSS (Direct Sequence Spread Spectrum)		
ADC INPUTS	(4) 10-bit ADC inputs		
DIGITAL I/O	15		
ANTENNA OPTIONS	Through-hole: PCB antenna, U.FL connector, RPSMA connector, or integrated wire SMT: RF pad, PCB antenna, or U.FL connector		
OPERATING TEMPERATURE	-40° C to 85° C (-40° F to 185° CF)		
DIMENSIONS (L X W X H) AND WEIGHT	Through-hole: 2.438 x 2.761 cm (0.960 x 1.087 in) SMT: 2.199 x 3.4 x 0.305 cm (0.866 x 1.33 x 0.120 in)		
PROGRAMMABILITY			
MEMORY	N/A	32 KB flash / 2 KB RAM	
CPU/CLOCK SPEED	N/A	HCS08 / up to 50.33 MHz	
NETWORKING AND SECURITY			
PROTOCOL	ZigBee PRO 2007, HA-Ready with support for binding/multicasting		
ENCRYPTION	128-bit AES		
RELIABLE PACKET DELIVERY	Retries/Acknowledgements		
IDS	PANID and addresses, cluster IDs and endpoints (optional)		
CHANNELS	16 channels		
POWER REQUIREMENTS			
SUPPLY VOLTAGE	2.1 to 3.6 V		
TRANSMIT CURRENT	33 mA @ 3.3 VDC / 45 mA boost mode	47 mA @ 3.3 VDC / 59 mA boost mode	
RECEIVE CURRENT	28 mA @ 3.3 VDC / 31 mA boost mode	42 mA @ 3.3 VDC / 45 mA boost mode	
POWER-DOWN CURRENT	<1 µA @ 25° C (77° F)	1.5 µA @ 25° C (77° F)	

Figura 37. Especificaciones técnicas XBee PRO S2C

Lenguajes, librerías o implementaciones firmware:

- Al ser compilado en c permite cargar archivos generados con implementaciones de MicroPython, se provee un conjunto de funcionalidades en el siguiente enlace.
- XBee desarrollo es una aplicación multiplataforma gratuita llamada XCTU, diseñada para interactuar con módulos Digi RF a través de una interfaz gráfica fácil de usar. Incluye nuevas herramientas que facilitan la instalación, configuración y prueba. Ejemplo - [Descripción de implementación para una WSN](#)

En cuanto kit de desarrollo tenemos las placas:

- [XBEE-MP-TH](#) : Es una placa de desarrollo que permite programar hasta 6 módulos de manera simultánea. Permite proveer actualizaciones de Firmware de manera automática en los dispositivos.
- [XBIB-CU-TH](#): Es una placa de desarrollo que permite programar un módulo directamente desde un PC.

Costo:\$140.000 +\$16.000 Adaptador USB

4.4.4 RASPBERRY PI ZERO W

Es una versión mejorada la Raspberry Pi Zero, debido a su tamaño, costo y capacidades de cómputo es un buen candidato para constituir un nodo en una red inalámbrica de sensores. Este componente está construido de la siguiente forma

- 802.11 b/g/n wireless LAN.
- Bluetooth® 4.1
- Bluetooth Low Energy (BLE)
- Procesador single-core CPU de 1GH
- 512MB RAM
- Puertos Mini HDMI y USB On-The-Go
- Alimentación por Micro USB
- HAT-compatible 40-pin header pins
- Video compuesto

- Conector CSI para camera
- Alimentación 5V 2.5A

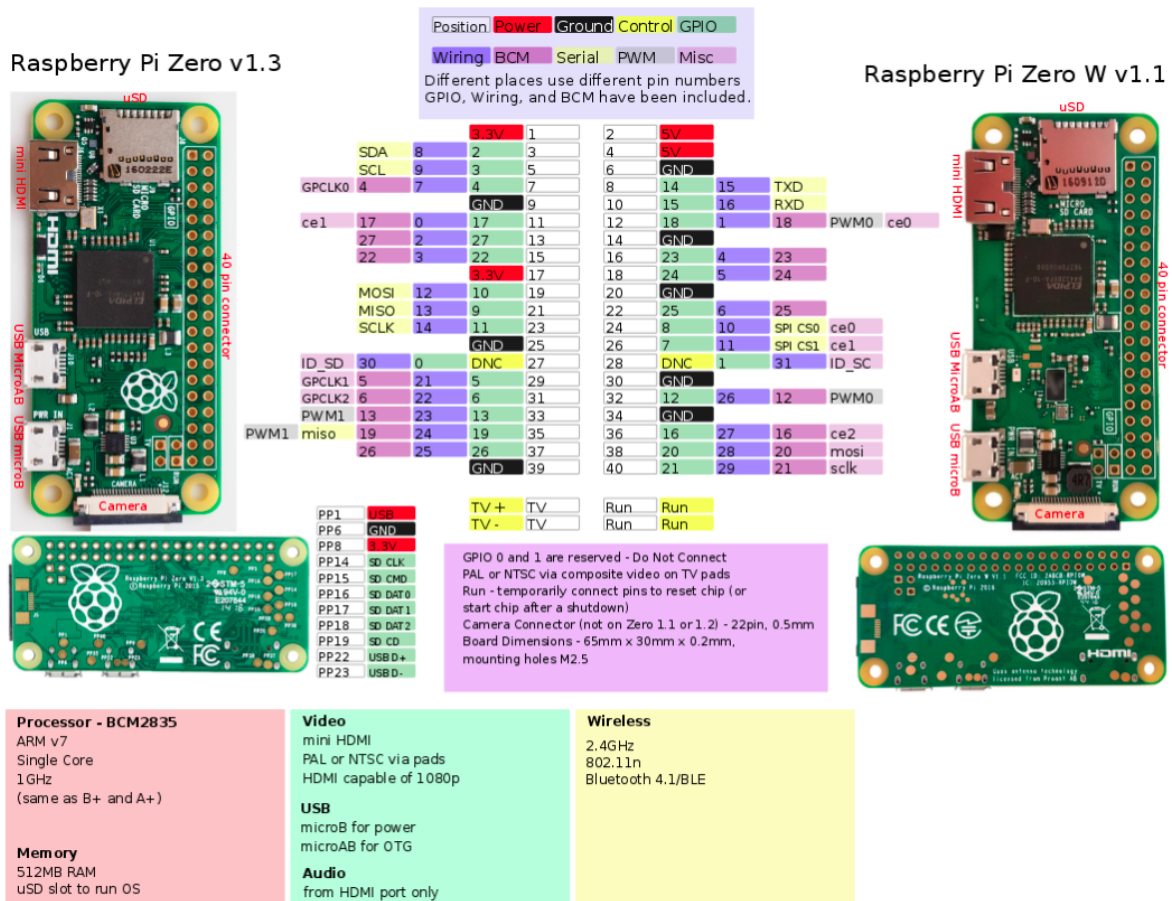


Figura 38.Review Raspberry Pi Zero

Los productos raspberry permiten cargar un SO ya que cuentan con las características de un computador personal. Razón por lo cual puede ser trabajado bajo cualquier lenguaje computacional.

Costo: \$48.000

4.4.5 RASPBERRY PI PICO

Es una versión diseñada para realizar aplicaciones mediante Micro Python. Cuenta con las siguientes características (datasheet):

- Microcontrolador: RP2040 diseñado por Raspberry Pi.
- Procesador Dual-core ARM Cortex M0+ a 133 MH
- 264kB de SRAM y 2MB de memoria Flash.
- Soporte USB 1.1 Host y Device.
- Soporte de bajo consumo "Sleep"
- Programación Drag & drop como dispositivo de almacenamiento masivo por USB.
- 26 pines GPIO.
- LED integrado programable.
- 2x SPI, 2x I2C, 2x UART, 3x 12-bit ADC, 16x canales PWM.
- Reloj y timer interno
- Sensor de temperatura integrado.
- FPU de coma flotante.
- 8 x pines PIO para máquinas de estados

El producto cuenta con varios estado de implementación, dentro de los cuales se destacan los modos DORMANT y SLEEP, su consumo está por debajo de las presentaciones habituales raspberry Pi, sin embargo carece de una interfaz inalámbrica que le permita ser parte de una red inalámbrica, debido a su precio y capacidad computación lo convierten en una excelente opción para enriquecer las capacidades de cómputo.

Costo: \$24.000

4.5 HERRAMIENTA PARA LA CONSTRUCCION DE ARCHIVOS DE SIMULACION E IMPLEMENTABLES.

Para la creación de código, cada nodo hace uso de una SDK que define la implementación concreta usada en dispositivos físicos y una implementación lógica usada para simulación.

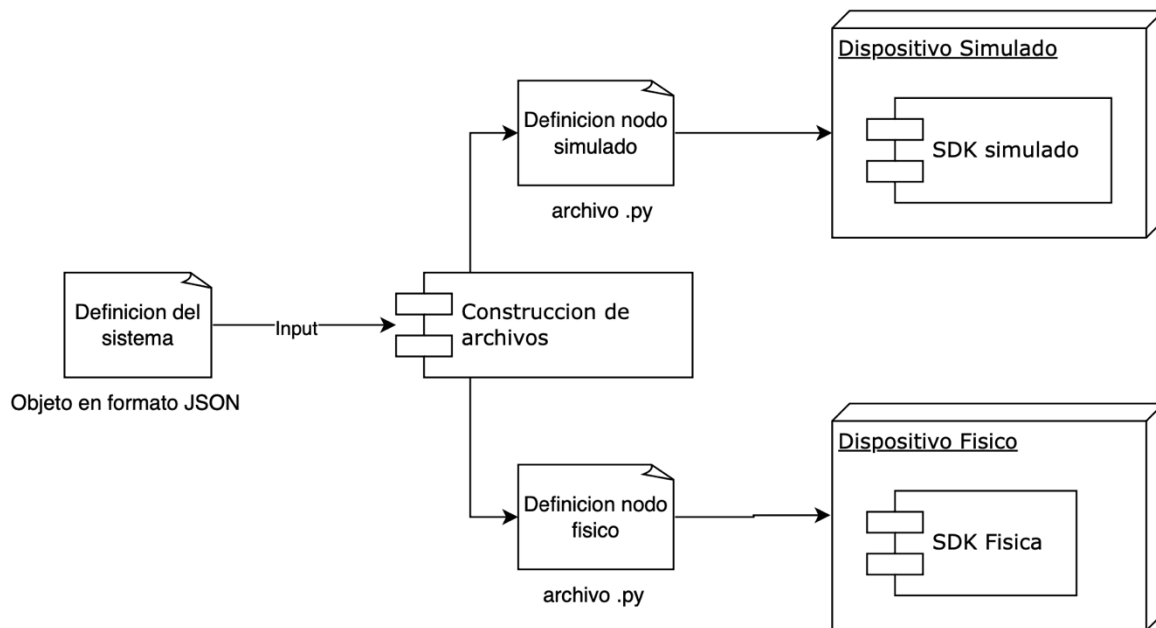


Figura 39. Diagrama de componentes para la creación de código

El objeto en formato JSON que describe el sistema tiene la siguiente estructura:

```
{
  > "allElements": [...],
  > "relationNode": [...],
  > "network": {"type": "Wireless"...}
}
```

Figura 40. Estructura objeto JSON definición del sistema

Las principales tags son "allElements", "relationNode", "network"

4.5.1 ESTRUCTURA "ALLELEMENTS" OBJETO JSON

La categoría "allElements", agrupa cada uno de los elementos de una red HiLes, a continuación la definición del bloque estructural que define el nodo de Humedad:

```

{
  "id": "1",
  "type": "structural_block",
  "name": "Nodo Humedad",
  "isGenerator": "false",
  "service": [...]
},

```

Figura 41. Objeto JSON bloque estructural HiLes nodo de humedad

a destacar el campo “isGenerator”, permite definir si el operador HiLes es generador de señales o no. Un generador de señales en un nodo es un componente que emite una señal, en el nodo de humedad, nos permite obtener el valor de humedad del ambiente. Con un generador de señal podemos modelar las características del dispositivo utilizado como sensor.

Así mismo el tag de “servicie” permite definir como el nodo interactúa con los demás nodos de la red, definiendo el puerto y los operadores de salida y entrada, como se ilustra a continuación:

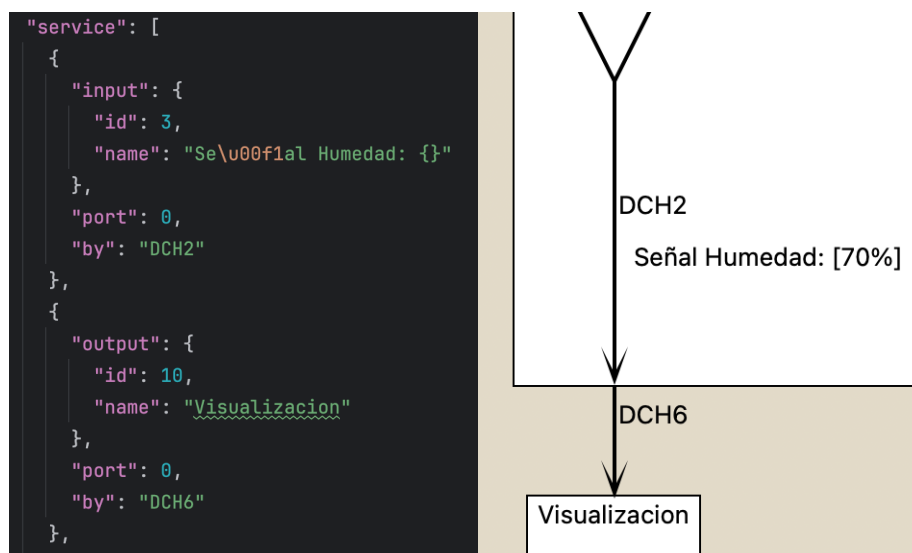


Figura 42. Definición de servicios en Objetos HiLes

4.5.2 ESTRUCTURA “RELATIONNODE” OBJETO JSON

La categoría “relationNode” agrupa los elementos HiLes que conforman cada nodo, la siguiente es la definición para el nodo de Humedad:

```
"relationNode": [  
  {  
    "name": "Nodo_Humedad",  
    "id": 1,  
    "operators": [2, 3, 4, 5, 6, 7, 8, 9],  
    "time_to_sleep": 2,  
    "netPetri": {  
      "name": "NET_P1",  
      "places": [7, 8],  
      "transitions": [6, 9]  
    }  
  },  
]
```

Figura 43. Objeto JSON nodo de humedad campo “relationNode”

Para el nodo de humedad los tags “operators”, “places”, “transitions” son arreglos que contienen el id del operador HiLes que contiene el nodo.

```
{
  "name": "Nodo_Riego",
  "id": 11,
  "operators": [
    12
  ],
  "time_to_sleep": 0,
  "netPetri": {
    "name": "",
    "places": [],
    "transitions": []
  }
}
```

Figura 44. Objeto JSON nodo de humedad campo "relationNode"

Para el nodo de Riego únicamente se cuenta con el operador sample que tiene el id 12.

4.5.3 ESTRUCTURA "NETWORK" OBJETO JSON

La categoría "network", define el tipo de red a implementar en el sistema.

```
,
  "network": {
    "type": "Wireless",
    "system": "centralized"
  }
}
```

Figura 45. Estructura objeto JSON campo "network"

En este caso, la red a implementar será centralizada y la comunicación entre nodos será orquestada por un elemento de la red, que no será modelado y hará parte de la parametria

de la red, cada nodo de la red se conectara vía socket al y creara por nodo un hilo para transmitir información y otro para recibir.

El componente para la construcción de archivos permite generar archivos .py que pueden ser usados para simular el comportamiento del nodo y así mismo representan el código sintetizable que se cargara en el dispositivo, continuación el detalle del componente:

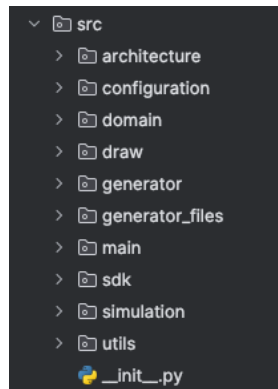


Figura 46. Estructura de paquetes para la construcción de archivos .py

Cada paquete se define a continuación:

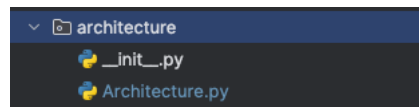


Figura 47. Paquete “architecture”

El paquete “architecture” permite agrupar todas las operaciones que definen la arquitectura como ejemplo, la creación de una Red de Petri, creación de operadores como sample o hold o la creación de cualquier canal lógico, discreto o continuo.

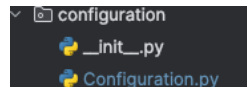


Figura 48. Paquete “configuration”

El paquete “configuration” agrupa las variables necesarias para la ejecución de los scripts como ejemplo, el número de pasos necesarios en la simulación o el nombre de las rutas estáticas en la creación de los archivos.

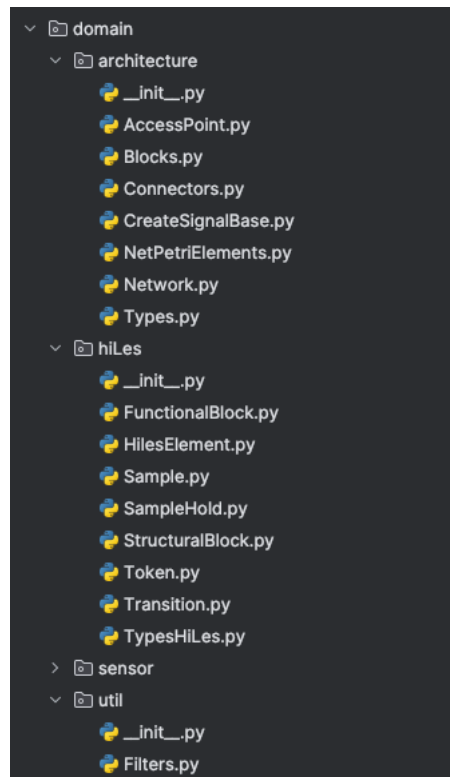


Figura 49. Paquete “domain”

Dentro del paquete “domain” se encuentra en el paquete “hiLes” que define los operadores HiLes, así mismo el paquete “architecture” define los elementos de la arquitectura como servicios o puertos, definición de la red o puntos de acceso, creación de señales. El paquete sensor agrupa los tipos de sensores, paquete útil define utilitarios utilizados por el paquete “domain”

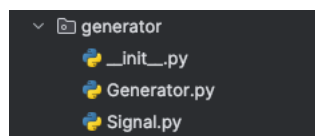


Figura 50. Paquete “generator”

El paquete “generator” es utilizado en la tapa de simulación al momento de generar señales.

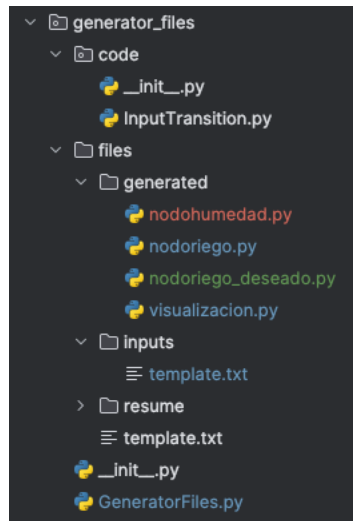


Figura 51. Paquete “generator_files”

El paquete “generator_files” establece las clases para generar archivos por su parte la carpeta archivos contiene los archivos generados y el “template” utilizado para la generación de código.

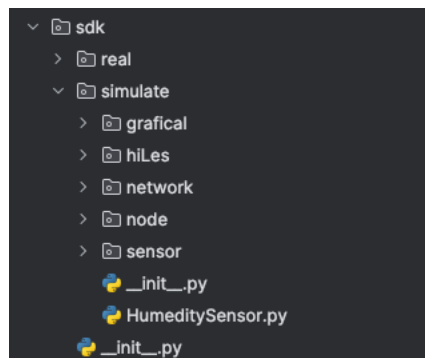


Figura 52. Paquete “sdk”

El paquete “sdk” contiene las clases a implementar en dispositivos reales en el paquete “real” y las clases a implementar en el simulado en el paquete “simulate”, cada paquete dentro de la SDK deberá contener la misma definición de clase, sin embargo a nivel de implementación cambiara de acuerdo a su contexto.

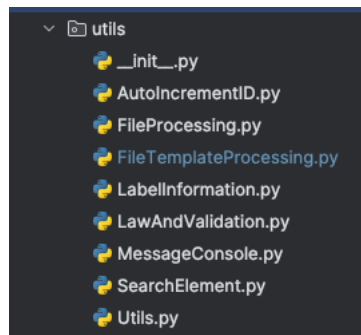


Figura 53. Paquete “utils”

El paquete es utilizado para largo de toda la solución para agrupar lógica transversal.

El siguiente es el código generado por la solución para el nodo de humedad es :

```
1 from src.sdk.simulate.node.Node import Node
2
3 node = Node(nameNode="Nodo_Humedad",time_to_sleep=2)
4 wireless = node.createGrid(type="Wireless")
5 humiditycontinuous2 = node.createSensor( type: "Humidity",mean=80,amplitude=10)
6 humiditylogical2 = node.createSensor( type: "Transmission",amplitude=1)
7 hilesOperator = node.createHilesOperator()
8 sample3 = hilesOperator.createSample(3,10,80)
9 funtional_block_4 = hilesOperator.createFunctionalBlock("<76")
10 funtional_block_5 = hilesOperator.createFunctionalBlock(">84")
11 netp1 = hilesOperator.createPetriNet([-1, 1])
12 netp1.createTransitions(2)
13 def operation():
14     while True:
15         output_humiditycontinuous2 = humiditycontinuous2.value()
16         output_humiditylogical2 = humiditylogical2.value()
17         output_sample3 = sample3.value(logical=output_humiditylogical2, continua=output_humiditycontinuous2)
18         output_funtional_block_4 = funtional_block_4.execute(output_humiditycontinuous2)
19         output_funtional_block_5 = funtional_block_5.execute(output_humiditycontinuous2)
20         output_netp1_0 = netp1.getToken(0)
21         output_netp1_1 = netp1.getToken(1)
22         netp1.getTransition(0).value(list([output_funtional_block_4.value]),output_netp1_0,output_netp1_1)
23         netp1.getTransition(1).value(list([output_funtional_block_5.value]),output_netp1_1,output_netp1_0)
24         wireless.emit(outputs=dict(Visualizacion=output_sample3,Nodo_Riego=output_netp1_1))
25
26     pass
27
28 if __name__ == '__main__':
29     operation()
```

Figura 54. Ejemplo de código generado para el nodo de humedad

La figura anterior detalla el código generado para el nodo de humedad, en etapa de simulación la SDK utilizada se define en la línea uno como :

```
1 from src.sdk.simulate.node.Node import Node
```

Figura 55. Importación sdk “simulate” en nodo etapa de simulación

Al momento de cargar este código en algún dispositivo la anterior línea del código deberá ser reemplazada por:

```
1 from src.sdk.simulate.node.Node import Node
```

Figura 56. Importación sdk “real” en nodo, para cargue en dispositivo físico

El código generado para el node de riego es:

```
from src.sdk.simulate.node.Node import Node

node = Node(nameNode="Nodo_Riego",time_to_sleep=0)
wireless = node.createGrid(type="Wireless")
hilesOperator = node.createHilesOperator()
sample12 = hilesOperator.createSample(12,0,0)
# OIYROBO_adlover *
def operation():
    while True:
        output_sample12 = sample12.value(logical=node.getVariable(name_node="visualizacion",port="0"),
                                         continua=node.getVariable(name_node="nodo humedad",port="1"))
        wireless.emit(outputs=dict(Visualizacion=output_sample12))
        pass

if __name__ == '__main__':
    operation()
```

Figura 57. Ejemplo de código generado nodo de riego

El repositorio que contiene el código necesario para generar el diagrama HiLeS y construir los archivos .py para la etapa de simulación y grabado en dispositivos físicos, se encuentra [aquí](#).

4.6 SIMULACION LOGICA DEL SISTEMA

La simulación del sistema permite analizar el comportamiento estableciendo una relación temporal, identificando dinámicamente la interacción de los elementos que componen el sistema.

4.6.1 SIMULACION LOGICA DEL NODO DE HUMEDAD

A continuación el comportamiento del sistema en el nodo de humedad:

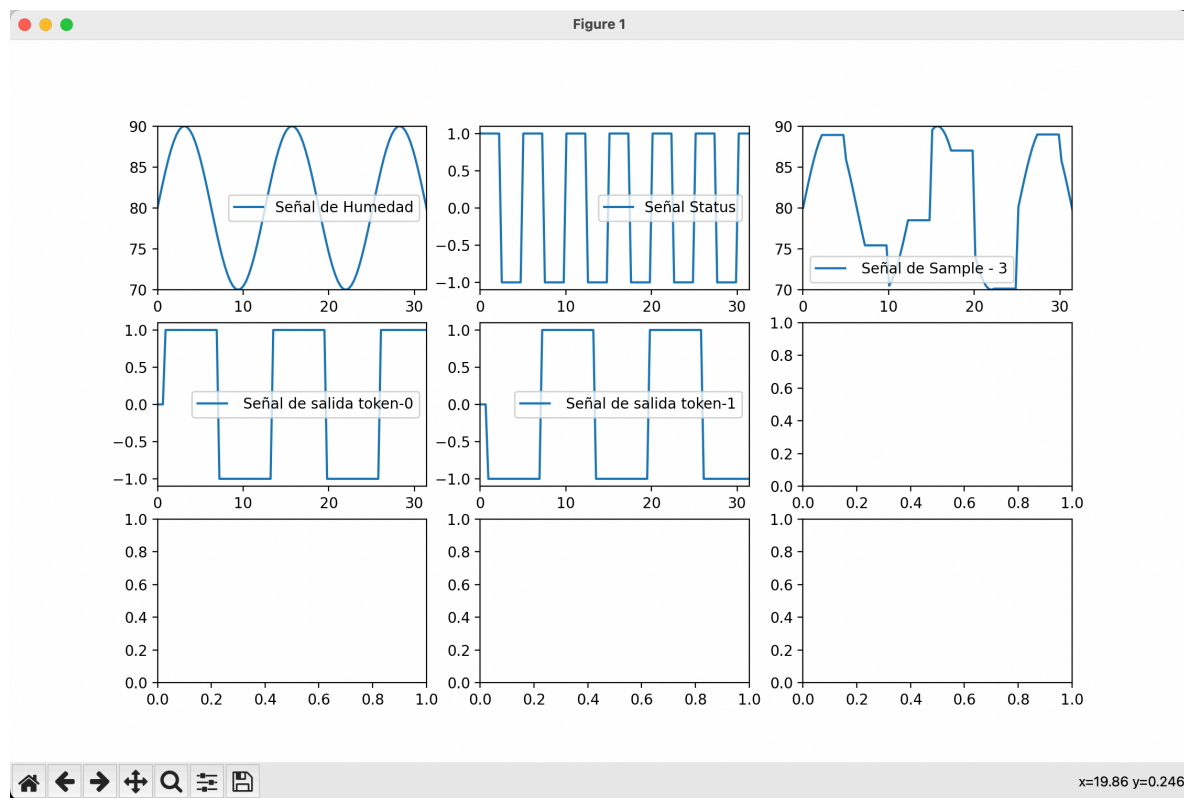


Figura 58. Simulación nodo de humedad

En la anterior figura se evidencian 5 señales. La “Señal de Humedad” permite simular los escenarios de variabilidad que pueda tener el valor de humedad en un periodo de tiempo. Esta forma de onda es sinusoidal y se define en la descripción textual de la arquitectura del sistema, en este caso varía entre 70% y 90%, a continuación el detalle de la implementación:


```
signalHumidity = Generator(typeSensorValues.humidity)
signalHumidity.setSignal(Signal( type: 'continuous', form: 'sin_10_80'))
signalHumidity.setSignal(Signal( type: 'logical', form: 'squart_1_0'))
signalHumidity.setDevice("DHT11Sensor(pin=5)")
nodoHumidity: StructuralBlock = architecture.structuralBlock(name=self.valueLabel.nodoHumidity,
                                                             coord=[38, 18, 786, 398], isEstructuralToNode=
                                                             time_to_sleep=2)
nodoHumidity.draw.sendBack()
humedad: StructuralBlock = architecture.structuralBlock(name=self.valueLabel.humedad,
                                                         position=[270, 50, 60], generator=signalHumidity)
```

Figura 59. Representación textual generador de señal en nodo de humedad

El string “sin_10_80”, representa la forma de onda sinusoidal con promedio de 80% y amplitud de 10%.

La “Señal Status” se construye con el string ‘squart_1_0’, lo cual representa la forma de onda cuadrada con amplitud 1 y promedio 0.

La señal “Señal de Sample-3”, es el muestreo de la “Señal de Humedad” a partir del estado de la “Señal Status” y representa la salida del operador Sample en el nodo de humedad.

La “Señal de salida token - 0”, muestra el comportamiento dinámico de la transición del token en P1, de la misma forma, la “Señal de salida token - 1” muestra el comportamiento para el token P2. Estas dos señales nos ayudan a identificar cuando el nodo de humedad le indicara al nodo riego que encienda el actuador. Su comportamiento esta alineado a la red de Petri construida con las reglas funcionales descritas a partir de los bloques funcionales en el nodo de humedad.

4.6.2 SIMULACION LOGICA DEL NODO DE RIEGO

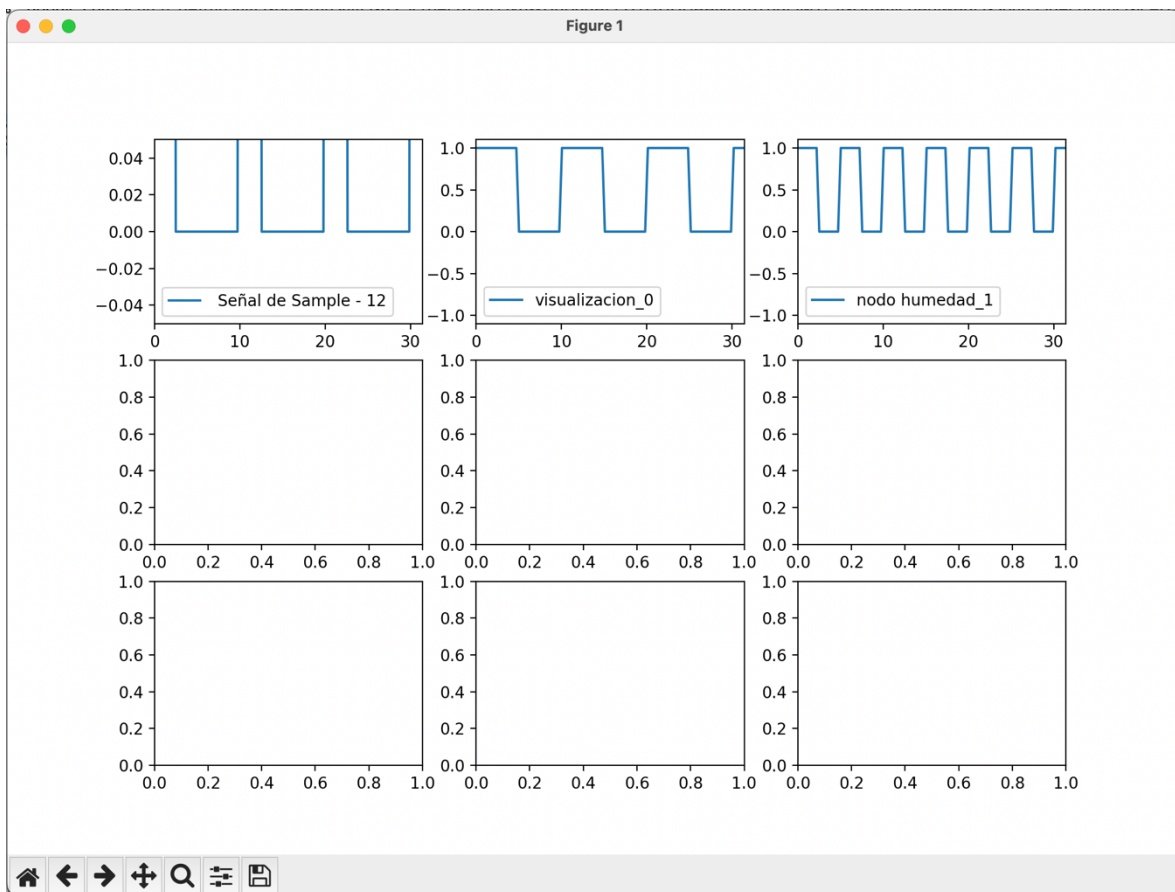


Figura 60. Simulación nodo de riego

En la anterior figura se evidencian 3 señales. La “Señal de Sample - 12” describe el comportamiento temporal de la señal que llegara al nodo de visualización desde el nodo de riego y es la salida del operador Sample, en donde 1 representa el estado de encendido del actuador de riego.

La señal “visualizacion_0” muestra la solicitud de “status” desde el nodo de visualización al nodo de riego.

La señal “nodo humedad_1” representa la señal de encendido de riego enviada desde el nodo de humedad al nodo de riego. Se evidencia que la señal de salida del operador de Sample estará en 1 cada vez que llegue una solicitud de “status” y el estado de la señal de encendido sea 1.

Al igual que en el nodo de humedad, la forma de onda de solicitud de “status” que genera el nodo de visualización es parametrizable en la descripción textual del sistema de la siguiente manera:

```
signalStatusVisualization = Generator(type=typeSensorValues.transmission)
signalStatusVisualization.setSignal(Signal( type: 'logical', form: 'square_1_0'))
visualizacion = architecture.structuralBlock(name=self.valueLabel.display,
                                             position=[84, 500, 90], generator=signalStatusVisualization,
                                             isEstructuralToNode=True)
```

Figura 61. Representación textual señal “status” en el nodo de visualización

4.7 VALIDACION DEL SISTEMA EN DISPOSITIVOS FISICOS

La validación en dispositivos físicos se realizara con el componente ESP8266, cada nodo (humedad y riego) será implementado en un componente. Para comprobar el comportamiento del sistema, se compara el resultado de la simulación lógica con el comportamiento del nodo real.

La siguiente imagen describe el montaje realizado para los nodos de humedad y visualización.

En caso del nodo de humedad usa el sensor de humedad DHT11, el nodo de visualización utiliza un led como indicador de encendido de riego.

De acuerdo al diagrama de requerimientos para solventar el requerimiento “Mantener el índice de humedad de la planta en 80% +- 4%”:

1. Estado inicial con el valor de humedad en 70% señal de riego en estado true.

Nombre de la Señal	Valor Simulado	Valor en dispositivos
CCH2 – Señal de humedad	70	Capturada en nodo de humedad:73
CCH8 – Señal de encendido nodo riego	true	Capturada en nodo de riego: true

Tabla 3. Validación de requerimiento estado uno

2. Emisión de señal de transmisión en sensor de humedad lo cual produce que se emita al nodo de visualización el valor de humedad al 70%.

Nombre de la Señal	Valor Simulado	Valor en dispositivos
DCH6 – paquete de humedad	70	Capturada en nodo de visualización:69

Tabla 4. Validación de requerimiento estado dos

3. Valor de humedad cambia a 79% la señal de riego se mantiene en estado true.

Nombre de la Señal	Valor Simulado	Valor en dispositivos
CCH2 – Señal de humedad	79	Capturada en nodo de humedad:80
CCH8 – Señal de encendido nodo riego	true	Capturada en nodo de riego: true

Tabla 5. Validación de requerimiento estado tres

4. El nodo de visualización emite una solicitud de “status”, con lo cual el nodo de riego envía su estado actual en true.

Nombre de la Señal	Valor Simulado	Valor en dispositivos
LCH1 – Solicitud de Status Nodo de visualización	true	Capturada en nodo de riego: true
DCH4 – paquete de encendido nodo riego	true	Capturada en nodo de visualización: true

Tabla 6. Validación de requerimiento estado cuatro

5. Valor de Humedad cambia a 86%, la señal de riego cambia a estado false.

Nombre de la Señal	Valor Simulado	Valor en dispositivos
CCH2 – Señal de humedad	86	Capturada en nodo de humedad:87
CCH8 – Señal de encendido nodo riego	false	Capturada en nodo de riego:

Tabla 7. Validación de requerimiento estado cinco

6. Se emite una señal de transmisión por parte del sensor de humedad en valor a 86% y se emita solicitud de status desde el nodo de visualización al nodo de riego.

Nombre de la Señal	Valor Simulado	Valor en dispositivos
CCH2 – Señal de humedad	86	Capturada en nodo de humedad:88
DCH4 – Paquete de encendido nodo riego	false	Capturada en nodo de visualización: false
CCH8 – Señal de encendido nodo riego	false	Capturada en nodo de riego: false
LCH1 – Solicitud de Status Nodo de visualización	true	Capturada en nodo de riego: true

Tabla 8. Validación de requerimiento estado seis

7. Valor de Humedad cambia a 60% y la señal de riego cambia a estado false.

Nombre de la Señal	Valor Simulado	Valor en dispositivos
CCH2 – Señal de humedad	60	Capturada en nodo de humedad:62
CCH8 – Señal de encendido nodo riego	true	Capturada en nodo de riego: true

Tabla 9. Validación de requerimiento estado siete

5. CONCLUSIONES

- HiLes es un especificación de alto nivel que permite describir el comportamiento de una red inalámbrica de sensores, estableciendo sus nodos e interacción entre los mismos.
- Los bloques estructurales de HiLes permiten definir nodos de manera abstracta como por ejemplo el nodo de visualización o nodos en forma específica como por ejemplo el nodo de humedad.
- Los bloques funcionales de HiLes permiten representar operación matemáticas que modifiquen el comportamiento de un nodo en una red inalámbrica de sensores.
- Los operadores de Petri permiten representar el intercambio de información de manera dinámica utilizando los bloques funcionales como entrada.
- La arquitectura de una red inalámbrica de sensores puede ser modelada con HiLes, estableciendo parámetros de entrada como:
 - Tipo de red -> Wifi
 - Tipo de comunicación -> centralizada
 - Dispositivos físicos a utilizar -> ESP32
 - Definición de nodos con dispositivos de sensado o actuadores.
- Es posible construir código implementable en dispositivos que cumpla con los requerimientos funcionales del sistema.

6. PRINCIPALES APORTES

- Se construyo un metamodelo HiLes que puede ser utilizado para definir una representación textual de una WSN.
- Se realizo en Tkinter una implementación visual de una representación textual de HiLeS.
- Se realizo un programa que a partir de la descripción textual de una WSN permite obtener en formato JSON las característica de una WSN de acuerdo al metamodelo HiLeS.
- Se realizo un programa que a partir del Objeto JSON permite obtener el código en lenguaje Python para simular el comportamiento de un nodo y en lenguaje MicroPython para implementar el nodo en dispositivos ESP32.
- Se realizo una SDK que permite simular el comportamiento de un nodo.
- Se realizo una SDK que permite implementar en dispositivos ESP32 y lenguajes MicroPython el nodo de una WSN modelada en HiLeS.
- Se planteó un ejemplo de una WSN con tres nodos definiendo la representación textual del sistema en HiLeS, dos nodos fueron implementados en ESP32 en lenguaje MicroPython, el comportamiento de cada nodo se simulo obteniendo resultados satisfactorios.

7. RECOMENDACIONES Y TRABAJOS FUTUROS

- Crear una interfaz gráfica que permita interactuar con los operadores HiLes para definir una WSN, posteriormente vincular obtener un descripción en formato JSON para realizar la construcción de código.
- Construir código implementable en dispositivos que permita comunicación distribuida en una WSN.
- Realizar la implementación de código para otros protocolos ejemplo bluetooth o zigbee.
- Implementar SDK para dispositivos físicos como ESP-01 o ESP8266.

8. REFERENCIAS

- Alliance, Z. (2002). *ZigBee Wireless Networks and Transceivers*. Drew Gislason.
- Alliance., ©. 2. (05 de 08 de 2020). *ZigBee Alliance* . Obtenido de ZigBee Alliance : <https://zigbeealliance.org/solution/zigbee/>
- Alliance®, L. (31 de 08 de 2019). *lora-alliance.org*. Obtenido de lora-alliance.org: <https://lora-alliance.org/>
- Andes, U. d. (01 de 06 de 2019). <https://hiles.uniandes.edu.co/>. Obtenido de <https://hiles.uniandes.edu.co/>: <https://hiles.uniandes.edu.co/>
- C. E. Gómez, J. C. (2010). Heterogeneous systems verification on HiLeS Designer tool. *IECON 2010 - 36th Annual Conference on IEEE Industrial Electronics Society*, (págs. 132-137).
- Claudia, A. (2012). Hiles-PL:Un Framework para la construcción de líneas especializada en el diseño de sistemas Embebidos. *Hiles-PL:Un Framework para la construcción de líneas especializada en el diseño de sistemas Embebidos*. Bogota, Cundinamarca, Colombia.
- Community, S. (12 de 02 de 2019). *SigFox*. Obtenido de SigFox: <https://www.sigfox.com/en>
- D. Han, J. L. (2010). Design and implementation of smart home energy management systems based on ZigBee. *IEEE Transactions on Consumer Electronics*, (págs. 1417-1425).
- G Muñoz, S. R. (2000). HiLesMaker: Diseño de Especificaciones de Alto NivelAsistida por Computador. *Ingeniería*, 19 - 22.
- Gianni Pasolini, C. B. (2018). Smart City Pilot Projects Using LoRa and IEEE802.15.4 Technologies. *sensors*, 2-17.

- Greaves, D. J. (2016). System on Chip Design and Modelling. En D. J. Greaves, *System on Chip Design and Modelling*. USA.
- J Garbarino, A. E. (2011). Protocolos para redes inalámbricas de sensores. En A. E. J Garbarino, *Protocolos para redes inalámbricas de sensores* (págs. 21 - 85). Buenos Aires: Universidad de Buenos Aires.
- Jimenez, F. (2000). Tools and models for systems design and synthesis of MEMS based on asynchronous circuits. *Proceedings of IEEE International Conference on Industrial Technology 2000* (págs. 64-69). Goa, India: IEEE.
- Johan S., R. R. (2017). Similitudes y diferencias entre Redes de Sensores Inalámbricas e Internet de las Cosas: Hacia una postura clarificadora. *Revista Colombiana de Computación*, 58–74.
- K. Gill, S. Y. (2009). A zigbee-based home automation system. *IEEE Transactions on Consumer Electronics*, 422-430.
- LAAS-CNRS. (s.f.). HiLeS DESIGNER - Tool Manual. *HiLeS DESIGNER - Tool Manual* . Universidad de los Andes Departamento de Ingeniería Eléctrica y Electrónica.
- LoRa Alliance, I. (2017). LoRaWAN™ 1.0.3 Specification. *LoRaWAN™ 1.0.3 Specification*. LoRa Alliance.
- Nabeel Ahmed Malik, M. U.-R. (2017). Green Communications Techniques and Challenges. *EAI Endorsed Transactions on Energy Web and Information Technology*.
- Saleh, R. (2006). System-on-Chip: Reuse and Integration. *Proceedings of the IEEE*, 1050-1069.
- Shaza Hanif, A. M. (2018). Opportunistically Exploiting Internet of Things for Wireless Sensor Network Routing in Smart Cities. *Journal of Sensor and Actuator Networks*, 7 - 46.
- Srivastava, N. (2010). Challenges of Next-Generation Wireless Sensor Networks and its impact on Society. *Journal of Telecommunications*, 128-133.